

Universidad Católica
de Santiago del Estero
Departamento Académico Rafaela

Scouting Inteligente con Inteligencia Artificial aplicado al Fútbol Juvenil

Evaluación y predicción del potencial de desarrollo mediante análisis de datos

Trabajo Final de Grado

Universidad Católica de Santiago del Estero

Departamento Académico Rafaela

Ingeniería en Informática

Alumno: Solari, Pablo

Directores: Duarte, Jorge – Lovera, Maximiliano

Rafaela, Santa Fe, Argentina

Octubre de 2026

ÍNDICE

LISTA DE FIGURAS 5

LISTA DE TABLAS 7

RESUMEN 8

ABSTRACT 9

1. INTRODUCCIÓN 10

1.1 Descripción del problema 10

1.2 Objetivos 12

1.3 Antecedentes 13

1.3.1 Influencia del Big Data y la IA en el fútbol 13

1.3.2 Importancia del software en el fútbol formativo 14

1.4 Justificación 15

1.5 Alcance 15

2. MARCO TEÓRICO 18

2.1 Definición de términos 18

2.1.1 Aprendizaje automático aplicado al deporte 18

2.1.2 Análisis de datos en el deporte 18

2.1.3 Relevancia de la inteligencia artificial en el deporte 19

2.1.4 Ciencia de datos y Big Data 19

2.1.5 Analítica de datos 20

2.1.6 Fundamentos de modelado y evaluación 20

2.1.6.1 Redes neuronales feed-forward y clasificación binaria 20

2.1.6.2 Normalización y vector de características 20
2.1.6.3 Métricas de evaluación de clasificación.....20
2.1.6.4 Calibración, desbalance y fuga de información 21
2.1.6.5 Datos sintéticos y sesgos de edad y maduración.....21
2.2 Trabajos relacionados y contexto de uso.....21
3. METODOLOGÍA.....23
3.1 Enfoque de datos y modelado 23
3.1.1 Recolección y estructura de datos 23
3.1.2 Preprocesamiento y normalización 23
3.1.3 Entrenamiento y evaluación.....23
3.1.4 Definición de la variable objetivo.....24
3.2 Metodología de desarrollo 24
3.3 Ciclo de vida del software 25
3.4 Secuencia de actividades realizadas 25
4. ANÁLISIS Y DISEÑO 27
4.1 Arquitectura del sistema 27
4.1.1 Arquitectura general, capas y componentes 27
4.1.1.1 Decisiones de arquitectura.....29
4.1.2 Diseño de la base de datos.....30
4.1.3 Interfaz de usuario 31
4.2 Funcionalidades y componentes.....32
4.2.1 Módulos funcionales implementados 32
4.2.2 Modelo de evaluación deportiva.....32

4.2.3 Uso de PyTorch y Scikit-learn 33
4.2.3.1 Síntesis explicativa del flujo predictivo.....34
4.2.4 Seguridad y autenticación 35
4.3 Requisitos del sistema 35
4.4 Modelo de datos 36
4.5 Endpoints principales del MVP 37
4.6 Decisiones tecnológicas y alineación con el MVP.....39
4.7 Casos de uso.....39
5. DESARROLLO 43
5.1 Implementación del backend 43
5.2 Implementación del frontend.....44
5.3 Datos, edad y categoría juvenil.....45
5.4 Entrenamiento del modelo de aprendizaje automático.....45
5.5 Seguridad y robustez implementada.....47
5.6 Despliegue del sistema.....48
5.7 Problemas técnicos y decisiones de implementación 50
6. PRUEBAS Y RESULTADOS.....51
6.1 Plan de pruebas.....51
6.2 Resultados del MVP 51
6.2.1 Métricas de PlayerNet y calibración 53
6.2.2 Comparación con baseline.....54
6.2.3 Evidencia operativa del despliegue 55

6.2.4 Evidencia de integración continua 64
6.3 Limitaciones observadas en la validación 65
6.4 Discusión 66
6.5 Consideraciones éticas y protección de datos 68
7. CONCLUSIONES 69
7.1 Conclusiones principales 69
7.2 Cumplimiento de objetivos 70
7.3 Limitaciones 72
7.4 Lecciones aprendidas 73
7.5 Trabajo futuro 73
8. BIBLIOGRAFÍA 75
9. ANEXOS TÉCNICOS 78
9.1 Rutas principales 78
9.2 Variables de entorno relevantes 78
9.3 Glosario técnico 78
9.4 Reproducción mínima 81
10. ANEXOS VISUALES Y EVIDENCIA 84
10.1 Evidencia de Render 84
10.2 Integración continua y documentos técnicos 84
10.3 Limitaciones documentadas del MVP 86
10.4 Diagramas ampliados 87
10.5 Casos de uso ampliados 93

LISTA DE FIGURAS

Figura 4-1. Diagrama de componentes de TPScouting. 29

Figura 4-2. Diagrama de clases del modelo de dominio. 37

Figura 4-3. Diagrama de casos de uso general 40

Figura 4-4. Diagrama de casos de uso de gestión de jugadores. 42

Figura 4-5. Diagrama de casos de uso de análisis y decisión scout. 42

Figura 5-1. Secuencia de inferencia: score operativo de interfaz y referencia calibrada. 44

Figura 5-2. Secuencia de carga del panel general. 45

Figura 5-3. Arquitectura de despliegue en Render y mecanismos operativos del MVP. 49

Figura 6-1. Curva real de entrenamiento generada desde training_metadata.json. 55

Figura 6-2. Login del MVP en la verificación local. 57

Figura 6-3. Panel general con métricas por rol. 58

Figura 6-4. Listado paginado de jugadores con edad, categoría y potencial. 59

Figura 6-5. Ficha de jugador con historial, atributos y acciones CRUD en modales. 62

Figura 6-6. Vista de predicción con edad y ajuste combinado de historial y posición. 63

Figura 6-7. Comparador múltiple de jugadores. 64

Figura 6-8. Ejecución CI 38815440730 del paquete publicado en el repositorio de entrega. 65

Figura 10-1. Vista ampliada de la ejecución CI 38815440730 del repositorio de entrega. 86

Figura 10-2. Diagrama de componentes de TPScouting ampliado. 88

Figura 10-3. Diagrama de clases del modelo de dominio ampliado. 89

Figura 10-4. Diagrama de secuencia de inferencia ampliado. 90

Figura 10-5. Diagrama de secuencia del panel general ampliado. 91

Figura 10-6. Diagrama de despliegue en Render ampliado. 92

Figura 10-7. Diagrama de casos de uso general ampliado. 94

Figura 10-8. Diagrama de casos de uso de gestión de jugadores ampliado. 95

Figura 10-9. Diagrama de casos de uso de análisis y decisión scout ampliado. 96

LISTA DE TABLAS

Tabla 3-1. Secuencia de actividades realizadas. 26

Tabla 4-1. Capas y responsabilidades del MVP. 30

Tabla 4-2. Comparación entre motores relacionales. 31

Tabla 4-3. Dimensiones de evaluación deportiva. 33

Tabla 4-4. Requisitos funcionales del sistema. 36

Tabla 4-5. Entidades principales del modelo de datos. 37

Tabla 4-6. Endpoints principales del MVP. 38

Tabla 4-7. Decisiones tecnológicas. 39

Tabla 4-8. Casos de uso principales. 39

Tabla 5-1. Configuración verificada del modelo PlayerNet. 46

Tabla 5-2. Estado del despliegue en Render. 48

Tabla 6-1. Cobertura del plan de pruebas. 51

Tabla 6-2. Resultados de validación automatizada y CI. 53

Tabla 6-3. Métricas de PlayerNet calibrado y crudo en test. 54

Tabla 6-4. Comparación con baselines. 55

Tabla 6-5. Prueba real de disponibilidad del despliegue en Render. 56

Tabla 7-1. Cumplimiento de objetivos. 72

Tabla 9-1. Rutas principales. 78

Tabla 9-2. Variables de entorno relevantes. 78

Tabla 9-3. Glosario técnico. 80

RESUMEN

El seguimiento de futbolistas juveniles en clubes con recursos limitados suele depender de registros dispersos y criterios difíciles de comparar a lo largo del tiempo. Ante este problema, se diseñó y desarrolló TPScouting, un producto mínimo viable orientado a organizar y analizar información deportiva de jugadores de 12 a 18 años. La aplicación permite gestionar perfiles, atributos técnicos, estadísticas, evaluaciones, reportes, disponibilidad e historiales. También incluye un panel general, herramientas de comparación y una estimación de potencial producida por un modelo implementado en PyTorch. La solución se evaluó mediante datos sintéticos, pruebas automatizadas, métricas de clasificación, modelos de referencia y evidencias fechadas de la aplicación, el despliegue y la integración continua. Los resultados demuestran la factibilidad técnica del flujo comprendido entre el registro y la persistencia de datos y la presentación de una estimación en la interfaz. El sistema se plantea como apoyo para organizar información y hacer trazables los criterios de evaluación, sin sustituir el análisis de entrenadores y scouts.

Palabras clave: scouting juvenil; fútbol formativo; inteligencia artificial; aprendizaje automático; análisis de datos; PyTorch.

ABSTRACT

Youth player monitoring in resource-constrained football clubs often relies on scattered records and criteria that are difficult to compare over time. This final project designed and developed TPScouting, a minimum viable product for organizing and analyzing sporting information about players aged 12 to 18. The application manages profiles, technical attributes, statistics, assessments, reports, availability, and historical records. It includes a dashboard, comparison tools, and a potential estimate produced by a PyTorch model. The solution was evaluated with synthetic data, automated tests, classification metrics, baseline comparisons, and dated evidence of application, deployment, and continuous-integration behavior. The results establish the technical feasibility of the complete flow from data entry and persistence to the presentation of an estimate in the interface. They do not establish sporting validity or social impact because no real players or clubs participated. TPScouting is therefore presented as a decision-support prototype that complements, rather than replaces, the judgment of coaches and scouts.

Keywords: youth scouting; youth football; artificial intelligence; machine learning; data analysis; PyTorch.

1. INTRODUCCIÓN

1.1 Descripción del problema

El trabajo toma como contexto a los clubes de fútbol formativo con recursos limitados que necesitan registrar y consultar de manera sistemática la evolución de sus jugadores juveniles. El proyecto no midió la frecuencia ni la magnitud de este problema en instituciones reales; lo utiliza como punto de partida para evaluar una solución técnica acotada.

Dentro de ese contexto, la ausencia de un proceso uniforme de registro y análisis puede dificultar la recuperación de antecedentes, la comparación entre jugadores y el seguimiento de su evolución. El trabajo no cuantifica inversión, desempeño institucional ni valor económico generado.

Cuando los registros de entrenamientos, partidos y evaluaciones no se almacenan de forma estructurada, resulta más difícil recuperar la información, comparar jugadores y observar su evolución con criterios consistentes. Esta relación constituye la necesidad funcional abordada por el MVP, no un resultado de campo del estudio.

Un seguimiento incompleto puede limitar la detección oportuna de cambios en el rendimiento y dificultar la revisión de decisiones formativas. El proyecto no evaluó efectos sobre oportunidades deportivas, trayectorias profesionales ni sostenibilidad institucional.

Frente a esta situación, el proyecto evalúa la factibilidad técnica de integrar registros estructurados, visualizaciones y un modelo predictivo en un MVP de apoyo al scouting. El trabajo no mide mejoras reales de

objetividad, visibilidad ni toma de decisiones, ya que utiliza datos sintéticos y no fue validado con clubes, scouts o jugadores.

La incorporación de inteligencia artificial al análisis deportivo ofrece herramientas para organizar y comparar información. Plataformas como Wyscout centralizan video y datos para apoyar la observación, la comparación y el reclutamiento (Hudl, s. f.). Sin embargo, disponer de más datos no garantiza decisiones más precisas: los resultados dependen de la calidad del registro, del objetivo definido y de la revisión humana.

Problema de investigación: ¿Cómo pueden la ciencia de datos y la inteligencia artificial contribuir a optimizar la identificación, la evaluación y la proyección del potencial de jugadores juveniles en clubes con recursos limitados?

Una limitación relevante para clubes con recursos reducidos es la gestión sistemática de la información de sus jugadores. El proyecto aborda el problema de registro y consulta, pero no evalúa inversiones ni resultados institucionales.

Los clubes pueden generar información de entrenamientos, partidos y evaluaciones sin disponer de un medio uniforme para procesarla. Los registros manuales pueden dificultar la recuperación, comparación y trazabilidad de esa información.

Asimismo, la dificultad para identificar patrones, tendencias y señales de evolución limita la toma de decisiones estratégicas. Esto puede ocasionar que algunos talentos no sean reconocidos a tiempo y que los

recursos formativos se asignen sin criterios consistentes, afectando tanto a la sostenibilidad del club como a las oportunidades de desarrollo de los jugadores.

La pregunta de investigación se responde dentro del alcance técnico: la ciencia de datos permite centralizar, historizar y comparar registros, y la inteligencia artificial permite producir una estimación reproducible a partir de esos registros. El MVP demuestra que ambas capacidades pueden integrarse en una aplicación accesible. No demuestra que optimicen recursos, detecten mejor el talento ni mejoren decisiones deportivas reales; esas afirmaciones requieren validación longitudinal con instituciones y participantes reales.

1.2 Objetivos

Objetivo general: diseñar, implementar y verificar un MVP que centralice información de futbolistas juveniles de 12 a 18 años, permita consultar su evolución y comparar perfiles, e integre una estimación de potencial basada en datos sintéticos, con trazabilidad de datos, artefactos y pruebas.

- Implementar registros estructurados e historiales de atributos, estadísticas y evaluaciones con reglas verificables.
- Implementar filtros, comparadores y una estimación de potencial que permitan priorizar perfiles para revisión humana, sin afirmar eficacia deportiva real.
- Presentar en fichas y paneles información trazable que pueda ser utilizada como apoyo por entrenadores, scouts y directivos.

- Evaluar la factibilidad de una herramienta de bajo costo para clubes con recursos limitados; su efecto sobre la igualdad de oportunidades queda fuera de la validación realizada.
- Verificar que el MVP organiza y recupera información; la reducción de tiempos, costos o recursos requiere un estudio posterior con usuarios reales.

Criterios de evaluación retrospectiva: la revisión del MVP considera funcionamiento de procesos, pruebas automatizadas, evidencia operativa y métricas del modelo. No se documentaron criterios de aceptación cuantitativos establecidos antes del experimento; por ello estos criterios no se presentan como hipótesis ni como umbrales a priori. El impacto sobre eficiencia, efectividad, igualdad de oportunidades o decisiones deportivas requiere validación posterior.

1.3 Antecedentes

El trabajo se sitúa en el uso de datos y aprendizaje automático como apoyo al análisis deportivo. La literatura muestra aplicaciones para valorar acciones, comparar perfiles y estudiar la evolución de jugadores, aunque la utilidad de cada enfoque depende del origen de los datos y de su contexto de validación (Pappalardo et al., 2019; Decroos et al., 2019; Lacan, 2024).

1.3.1 Influencia del Big Data y la IA en el fútbol

- El análisis de eventos, estadísticas e historiales permite construir evaluaciones multidimensionales y comparar perfiles de

jugadores con criterios consistentes (Pappalardo et al., 2019; Decroos et al., 2019).

- Plataformas profesionales como Wyscout centralizan video y datos deportivos para apoyar procesos de observación, comparación y reclutamiento (Hudl, s. f.).
- La literatura reciente también explora modelos de aprendizaje automático para detectar perfiles de alto potencial y pronosticar la evolución futura de los jugadores. Sus resultados dependen de la calidad de los datos, la definición del objetivo y el contexto de aplicación (Lacan, 2024; van Arem et al., 2025).

1.3.2 Importancia del software en el fútbol formativo

- En instituciones con recursos limitados, los registros pueden permanecer distribuidos entre planillas y observaciones no normalizadas. En esas condiciones resulta difícil conservar la trazabilidad y comparar evaluaciones; el trabajo no cuantifica la frecuencia ni el efecto deportivo de esa situación.
- La observación experta aporta contexto cualitativo indispensable, pero cuando los registros no están estandarizados resulta difícil comparar evaluaciones, conservar trazabilidad y revisar decisiones.
- TPScouting evalúa la factibilidad de centralizar esos registros mediante componentes de software de uso abierto. La reducción de una brecha tecnológica o la mejora de decisiones no fueron medidas y se mantienen como posibles líneas de validación futura.

1.4 Justificación

La justificación técnica consiste en integrar, dentro de un único MVP, registros estructurados, visualizaciones, comparadores y una estimación reproducible. La evaluación comprueba el funcionamiento del flujo, pero no permite afirmar mejoras en la precisión del scouting ni igualdad entre clubes.

- Integración técnica: analítica de datos y aprendizaje automático aplicados a un flujo de scouting juvenil.
- Aplicación funcional: registro, consulta y comparación de información deportiva en un MVP web.
- Accesibilidad potencial: uso de componentes abiertos y reproducibles; la adopción y el impacto social no fueron evaluados.
- Caso práctico: demostración de factibilidad mediante un MVP web integrado con un módulo de predicción.
- Visualización y comunicación de datos: mediante paneles, gráficos, comparadores, vistas imprimibles y el registro de reportes scout, el sistema facilita la interpretación y comunicación de la información entre entrenadores, directivos y otras partes interesadas. *La exportación formal de informes y archivos PDF se mantiene como trabajo futuro.*

1.5 Alcance

El alcance del proyecto comprende un MVP académico de scouting juvenil que permite registrar jugadores, cargar atributos y estadísticas, consultar fichas, comparar perfiles y calcular una estimación de potencial mediante un modelo de aprendizaje automático integrado al backend Flask.

La solución se enfoca en una institución o entorno de evaluación acotado, con datos sintéticos para demostrar el flujo completo sin depender de proveedores externos de datos deportivos.

Incluye:

• Autenticación por usuario y contraseña, roles básicos y protección de rutas sensibles.

• Gestión de jugadores, staff, atributos, rendimiento, disponibilidad, evaluaciones físicas, reportes y comparadores.

• Carga individual y carga masiva por CSV compatible con Excel, con previsualización y validación por fila.

• Panel general con métricas adaptadas al rol del usuario.

• Inferencia de potencial con una red neuronal denominada PlayerNet en PyTorch, usando features derivadas de atributos, edad, posición e historial disponible.

• Pruebas automatizadas con pytest, cobertura en Integración Continua y “smoke real” sobre Render.

Queda fuera del alcance del MVP:

• Análisis automático de video, tracking GPS, integración con Wyscout/Hudl u otros proveedores comerciales.

• Gestión multi-club, módulos financieros, contratos, valuación económica o scouting internacional distribuido.

• Entrenamiento con datos reales longitudinales de clubes, ya que el MVP utiliza datos sintéticos para reproducibilidad académica.

• Arquitectura productiva multi-instancia con cache distribuido, rate limiting centralizado, auditoría completa y orquestación de pipelines.

Criterios mínimos de éxito observables:

• Ejecutar la suite automatizada sin fallos.

• Acceder al despliegue documentado, iniciar sesión y navegar el panel general, el listado, la ficha, la predicción y los comparadores.

• Mantener atributos dentro de escala 1-20 y potencial categorizado en bajo, medio y alto con umbrales documentados.

• Registrar evidencia real de métricas de entrenamiento, prueba real en Render y capturas de funcionamiento.

2. MARCO TEÓRICO

2.1 Definición de términos

2.1.1 Aprendizaje automático aplicado al deporte

El aprendizaje automático (machine learning, ML) es una rama de la inteligencia artificial que permite a las máquinas aprender a partir de la experiencia sin ser programadas de manera explícita para cada tarea. Se centra en el desarrollo de algoritmos capaces de procesar datos y producir predicciones o decisiones basadas en ellos (Hastie et al., 2009; Goodfellow et al., 2016).

Aplicación en el deporte: el aprendizaje automático puede apoyar tareas de evaluación de rendimiento, valoración de acciones y detección de perfiles, siempre que el objetivo y los datos estén definidos de manera verificable (Pappalardo et al., 2019; Decroos et al., 2019; Lacan, 2024). En este trabajo su uso se limita a una clasificación experimental sobre datos sintéticos.

2.1.2 Análisis de datos en el deporte

El análisis de datos deportivos permite resumir eventos, comparar perfiles y conservar evidencia de las evaluaciones. Los enfoques de PlayeRank y VAEP muestran que la utilidad de una métrica depende de la función asignada al jugador y del contexto de cada acción (Pappalardo et al., 2019; Decroos et al., 2019).

2.1.3 Relevancia de la inteligencia artificial en el deporte

En este trabajo, la inteligencia artificial se utiliza en una tarea acotada de clasificación sobre datos sintéticos. No se evaluaron recomendaciones tácticas, programas de entrenamiento ni efectos sobre el rendimiento real. Los antecedentes de scouting predictivo se consideran comparaciones metodológicas y no pruebas de validez para TPScouting (Lacan, 2024; van Arem et al., 2025).

El uso de información deportiva de menores plantea riesgos de privacidad, seguridad y sesgo. La edad relativa, la maduración y el contexto pueden afectar tanto los registros como su interpretación; por ello se requieren transparencia y revisión humana (Cobley et al., 2009). Las obligaciones para un uso real se desarrollan en la Sección 6.5.

2.1.4 Ciencia de datos y Big Data

La ciencia de datos combina procedimientos de obtención, preparación, análisis y comunicación de datos. El aprendizaje automático aporta modelos para tareas predictivas, cuya validez depende de la separación entre entrenamiento y evaluación (Hastie et al., 2009; Goodfellow et al., 2016).

Aunque Big Data suele describir problemas de volumen, velocidad y variedad, TPScouting no procesa datos masivos. El proyecto utiliza un conjunto sintético de 20.000 jugadores para comprobar un flujo técnico; por ello se emplean los términos ciencia de datos y analítica sin atribuir al MVP una arquitectura de Big Data.

2.1.5 Analítica de datos

La analítica de datos comprende la preparación, exploración, síntesis y visualización de información para responder preguntas definidas. En TPScouting se aplica al historial de atributos, las estadísticas, los reportes y las comparaciones. El componente predictivo se trata por separado porque requiere entrenamiento y evaluación de un modelo.

2.1.6 Fundamentos de modelado y evaluación

2.1.6.1 Redes neuronales feed-forward y clasificación binaria

Una red neuronal feed-forward transforma un vector de características en una salida numérica mediante capas densas y funciones de activación. En problemas de clasificación binaria, la salida puede interpretarse como un score que luego se transforma en probabilidad o categoría mediante un umbral definido.

2.1.6.2 Normalización y vector de características

La normalización permite llevar atributos heterogéneos a rangos comparables antes de ingresarlos al modelo. En scouting juvenil, los atributos pueden cargarse en una escala ordinal 1-20 y transformarse internamente a valores normalizados para el entrenamiento e inferencia.

2.1.6.3 Métricas de evaluación de clasificación

Para evaluar modelos de clasificación se emplean métricas complementarias. Accuracy mide aciertos globales, F1 combina precisión y recall, ROC-AUC evalúa separación entre clases y PR-AUC resulta especialmente útil cuando la clase positiva es minoritaria.

2.1.6.4 Calibración, desbalance y fuga de información

La discriminación y la calibración responden a preguntas diferentes: ROC-AUC evalúa el orden de los casos, mientras que una probabilidad calibrada busca coherencia entre frecuencias observadas y probabilidades estimadas. La calibración isotónica debe ajustarse fuera del conjunto de test para conservar una evaluación final independiente (Niculescu-Mizil & Caruana, 2005). Con una clase positiva minoritaria, PR-AUC complementa ROC-AUC porque expone la relación entre precisión y recall para la clase de interés (Saito & Rehmsmeier, 2015). Cualquier estadístico calculado con todo el conjunto antes del split puede transferir información de validación o test y debe declararse como riesgo de fuga (Kaufman et al., 2012).

2.1.6.5 Datos sintéticos y sesgos de edad y maduración

Los datos sintéticos permiten probar el flujo técnico sin exponer datos personales, pero su distribución y su etiqueta reflejan reglas de generación y no evidencia observada en futbolistas. Además, agrupar juveniles por edad cronológica puede favorecer sistemáticamente a quienes nacieron antes dentro del año de selección o maduraron antes. La literatura identifica este efecto de edad relativa como un sesgo relevante en el desarrollo deportivo (Cobley et al., 2009). Por ello, edad, maduración y contexto deben analizarse antes de cualquier uso real del score.

2.2 Trabajos relacionados y contexto de uso

Los trabajos relacionados pueden agruparse en cuatro líneas. PlayeRank propone una evaluación multidimensional y sensible al rol a partir de registros masivos de eventos de partido, y contrasta sus

resultados con valoraciones de scouts profesionales (Pappalardo et al., 2019). El enfoque VAEP valora las acciones individuales según su efecto sobre las probabilidades de marcar y recibir goles, incorporando el contexto de cada acción (Decroos et al., 2019).

En una línea próxima al scouting predictivo, Lacan (2024) utiliza una arquitectura de aprendizaje profundo por stacking para detectar jugadores de alto potencial sobre una base abierta. Van Arem et al. (2025) comparan modelos explicables para pronosticar la calidad y el valor futuro de futbolistas profesionales. TPScouting se diferencia por integrar captura, seguimiento histórico, visualización, reglas operativas e inferencia sobre datos sintéticos; no se plantea como competidor de plataformas profesionales.

3. METODOLOGÍA

3.1 Enfoque de datos y modelado

Se utilizan datos sintéticos para validar el flujo completo de captura, persistencia, entrenamiento e inferencia. Esta decisión favorece la reproducibilidad y evita depender de bases privadas de clubes o proveedores externos durante la evaluación académica, pero limita la validez externa de los resultados.

3.1.1 Recolección y estructura de datos

Los datos se organizan alrededor de jugadores juveniles, atributos técnicos, físicos, mentales, estadísticas de rendimiento, disponibilidad, reportes de scouting y evaluaciones físicas.

3.1.2 Preprocesamiento y normalización

Los atributos deportivos se cargan en escala 1-20. Para el modelo se transforman a representaciones numéricas compatibles con el preprocesador. Las posiciones se codifican como características cualitativas y se incorporan variables históricas cuando existen registros previos.

3.1.3 Entrenamiento y evaluación

El entrenamiento se realiza con semilla fija, partición estratificada train/validation/test y métricas de clasificación. El modelo operativo es el del trabajo (PlayerNet) en PyTorch. Scikit-learn se utiliza para preprocesamiento, partición de datos, métricas, regresión logística de referencia y calibración isotónica pero no constituye el motor principal de inferencia de la aplicación.

3.1.4 Definición de la variable objetivo

La corrida oficial no utiliza potential_label como variable objetivo ni incorpora potential_label, temporal_target_label o predicciones persistidas entre las 64 columnas de entrada (68 tras la codificación). El target efectivo es temporal_target_label, una etiqueta binaria sintética construida con señales futuras simuladas. La clase positiva se selecciona mediante un score temporal, controles de calidad y una cuota por cohorte de posición y edad. Por ello, la prevalencia de 1.597/20.000 (7,985 %) está fijada por el diseño sintético y no constituye una observación sobre futbolistas reales.

El score previo a los controles pondera crecimiento (0,15), nivel futuro (0,12), rendimiento (0,13), presión (0,13), consistencia (0,10), rol (0,09), disponibilidad (0,09), recuperación (0,09), evaluación scout (0,10) y breakout (0,14). Además, aplica una penalización por inestabilidad de 0,08. La distribución total registrada es de 1.597 casos positivos y 18.403 negativos sobre 20.000 jugadores sintéticos (7,985 %).

No se detectó una inclusión directa de la etiqueta ni de datos futuros entre las variables de entrada. Sin embargo, los cuantiles y las cuotas utilizados para construir el target se calculan sobre el conjunto completo antes de dividirlo en entrenamiento, validación y prueba. Esta decisión introduce un riesgo metodológico: las etiquetas de validación y prueba no son totalmente independientes de la distribución global, por lo que las métricas deben interpretarse como evidencia técnica preliminar.

3.2 Metodología de desarrollo

Se aplicó un enfoque incremental organizado en bloques pequeños y verificables. Cada cambio relevante se acompañó con pruebas focales,

una ejecución de la suite y documentación técnica. Se tomaron como referencia conceptos de iteración e inspección descritos en la Guía Scrum, pero no se aplicó Scrum de forma completa porque no se documentaron sus responsabilidades, eventos y artefactos (Schwaber & Sutherland, 2020).

3.3 Ciclo de vida del software

El ciclo se presenta como una secuencia de ingeniería de software adaptada al proyecto y no como evidencia de sprints formales (Pressman, 2010; Sommerville, 2011).

1. Análisis de requisitos y alcance del MVP.

2. Diseño de entidades, rutas, vistas y flujo de datos.

3. Implementación iterativa en Flask, SQLAlchemy, Jinja2, Bootstrap, Chart.js y PyTorch.

4. Pruebas automatizadas y validaciones manuales de flujos críticos.

5. Despliegue en Render con Gunicorn y PostgreSQL administrado para demo académica.

6. Documentación de limitaciones, evidencia y oportunidades de mejora.

3.4 Secuencia de actividades realizadas

Fase	Bloque	Actividad	Estado
1	Análisis y requisitos	Definición de alcance, actores, entidades y criterios de éxito.	Completado
2	Backend y persistencia	Modelos SQLAlchemy, rutas Flask, migraciones manuales y validaciones.	Completado

Fase	Bloque	Actividad	Estado
3	Frontend y UX	Vistas Jinja2, Bootstrap, modales CRUD, panel general, comparadores y responsive.	Completado
4	Machine Learning	Dataset sintético, entrenamiento, métricas, checkpoint y carga en inferencia.	Completado
5	Seguridad y robustez	CSRF, sesión, roles, secret obligatoria, rate limiting básico y errores controlados.	Completado para MVP
6	Testing y CI	pytest, pytest-cov, GitHub Actions, smoke visual opcional y smoke Render.	Completado
7	Documentación y entrega	Documentación del trabajo realizado y evidencia verificable.	Completado

TABLA 3-1. SECUENCIA DE ACTIVIDADES REALIZADAS.

4. ANÁLISIS Y DISEÑO

4.1 Arquitectura del sistema

4.1.1 Arquitectura general, capas y componentes

El proyecto utiliza una arquitectura web full-stack con Flask y Bootstrap, integrando persistencia relacional, análisis de datos y aprendizaje automático para apoyar la evaluación de jugadores juveniles.

Backend:

- Lenguaje: Python.
- Framework: Flask.
- Base de datos: SQLite para desarrollo local y PostgreSQL administrado en Render para la demo pública, ambos gestionados mediante SQLAlchemy (ORM). Se mantiene portabilidad hacia motores relacionales alternativos sin modificar la lógica de dominio principal.
- Bibliotecas de aprendizaje automático: PyTorch y scikit-learn.
- Almacenamiento: En desarrollo local los datos se pueden almacenar en SQLite; en el despliegue verificado se utiliza PostgreSQL en Render. La estructura registra jugadores, atributos, evaluaciones, disponibilidad, rendimiento y reportes a lo largo del tiempo.

- Frontend:

- Diseño de la interfaz: Basado en Bootstrap, con gráficos generados dinámicamente mediante Chart.js y tablas HTML con filtros y paginación.
- Interacción del usuario: Los usuarios pueden consultar datos de jugadores, visualizar proyecciones y analizar evolución mediante gráficos interactivos y comparadores.

La Figura 4-1 resume los componentes principales de la solución implementada: cliente web, aplicación Flask, blueprints funcionales, servicios auxiliares, persistencia y módulo de aprendizaje automático.

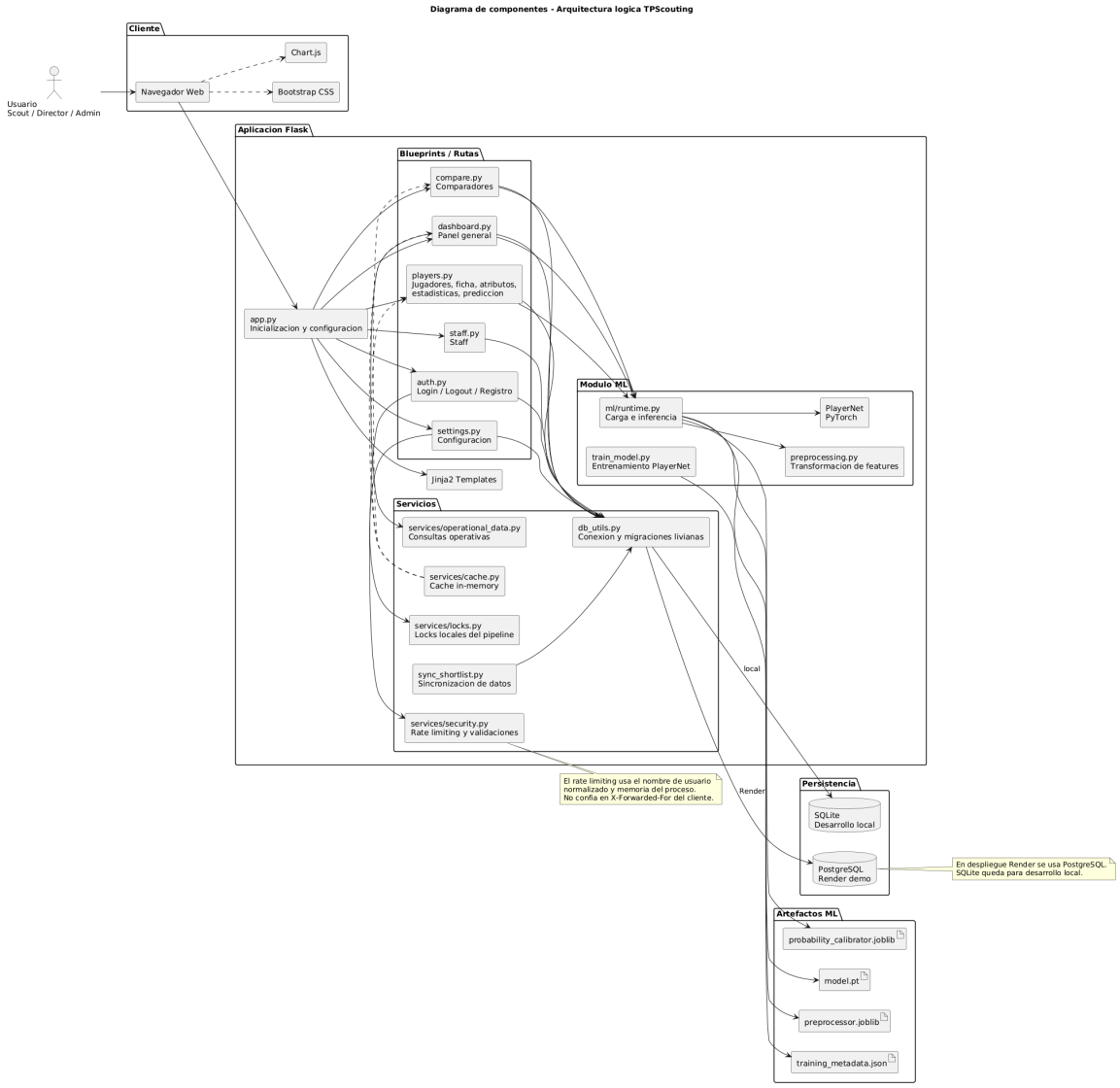


FIGURA 4-1. DIAGRAMA DE COMPONENTES DE TPSCOUTING.

4.1.1.1 Decisiones de arquitectura

Se seleccionó Flask porque la aplicación y el pipeline de aprendizaje automático utilizan Python y porque el renderizado server-side cubre el alcance del MVP (Pallets Projects, s. f.). SQLAlchemy mantiene un

mismo modelo de dominio sobre SQLite local y PostgreSQL en el despliegue (SQLAlchemy, s. f.). .NET Minimal APIs, Peewee, MySQL y SQL Server no fueron implementados ni comparados mediante benchmarks; por ese motivo no se les atribuyen ventajas o desventajas experimentales.

La arquitectura actual es una aplicación Flask server-side con Jinja2 y Bootstrap en frontend, SQLAlchemy como ORM, SQLite para desarrollo local y PostgreSQL en Render para demo pública. El backend fue modularizado parcialmente mediante blueprints por familia: auth, staff, players, dashboard, compare y settings. La inferencia ML se encapsula en módulos de runtime, manteniendo compatibilidad con los endpoints existentes.

Capa	Archivos principales	Responsabilidad
Presentación	templates/, static/	Vistas Jinja2, Bootstrap, Chart.js, CSS propio
Rutas	scouting_app/routes/	Blueprints por familia funcional
Servicios	scouting_app/services/	Cache, seguridad, locks y datos operativos
Dominio	models.py, player_logic.py	Entidades SQLAlchemy y reglas de negocio
ML	train_model.py, ml/runtime.py	Entrenamiento, checkpoint, carga e inferencia
Persistencia	SQLite/PostgreSQL	SQLite local; PostgreSQL en Render

TABLA 4-1. CAPAS Y RESPONSABILIDADES DEL MVP.

4.1.2 Diseño de la base de datos

- La base de datos se diseñó de forma relacional mediante SQLAlchemy. En desarrollo local y pruebas, el MVP utiliza SQLite por su simplicidad y portabilidad; el despliegue público en Render

utiliza PostgreSQL administrado para conservar los datos fuera del sistema de archivos efímero del servicio.

- SQLAlchemy permite mantener el mismo modelo de dominio en ambos entornos.
- El modelo implementa once entidades: Player, Match, Coach, Director, User, PlayerStat, PlayerAttributeHistory, PlayerMatchParticipation, ScoutReport, PhysicalAssessment y PlayerAvailability. Sus responsabilidades se resumen en la Tabla 4-5.

TABLA 4-2. COMPARACIÓN ENTRE MOTORES RELACIONALES.

4.1.3 Interfaz de usuario

- La interfaz se implementó con Bootstrap, plantillas Jinja2, CSS propio y Chart.js. Se priorizaron formularios, tablas, modales y gráficos compatibles con el flujo server-side del MVP.
- Una Landing Page con un login, que permite a los usuarios ingresar al sistema y acceder al panel de control donde pueden gestionar jugadores, ver estadísticas y generar proyecciones.
- Panel de Jugadores: Donde los usuarios pueden ver la lista de jugadores y acceder a su información detallada, incluyendo sus atributos y sus evaluaciones.
- Gráficos y Tablas:
    - Chart.js permite crear gráficos dinámicos e interactivos, visualizando la evolución de los atributos del jugador.

- Tablas HTML con filtros y paginación en backend para visualizar la información tabular de forma clara y operativa.

4.2 Funcionalidades y componentes

4.2.1 Módulos funcionales implementados

• Autenticación, registro de usuarios y control básico por roles.

• Gestión de jugadores, ficha completa, edición, baja y carga masiva CSV.

• Historial de atributos, rendimiento, partidos, disponibilidad, evaluaciones físicas y reportes scout.

• Panel general dinámico por rol: prioridades de scouting para scout/administrador y estado del plantel para director técnico.

• Comparador 1v1 y comparador múltiple con métricas visuales.

• Predicción de potencial y recomendaciones operativas en ficha de jugador.

4.2.2 Modelo de evaluación deportiva

La evaluación del jugador combina datos cuantitativos y cualitativos. Los atributos principales se registran en escala 1-20 y se agrupan en dimensiones técnicas, físicas, mentales y defensivas. El sistema no reemplaza el criterio del scout; aporta orden, trazabilidad y comparación consistente entre jugadores.

Dimensión	Variables representativas	Fuente / Tablas
Técnica	pace, shooting, passing, dribbling, technique, vision	Player e historial de atributos
Física	physical, estimated_speed, endurance, height_cm, weight_kg	Player y PhysicalAssessment
Defensiva	defending, tackling	Player e historial de atributos
Mental	determination, decision_making, tactical_reading, mental_profile, adaptability	Player y ScoutReport
Rendimiento	minutes_played, goals, assists, pass_accuracy, shot_accuracy, duels_won_pct, final_score	PlayerStat y participaciones

TABLA 4-3. DIMENSIONES DE EVALUACIÓN DEPORTIVA.

4.2.3 Uso de PyTorch y Scikit-learn

PyTorch se emplea para implementar PlayerNet (Paszke et al., 2019; PyTorch Foundation, s. f.). Scikit-learn aporta preprocesamiento,

partición, métricas, regresión logística y calibración isotónica (Pedregosa et al., 2011). La inferencia web utiliza PlayerNet como modelo operativo; la regresión logística se conserva como referencia experimental.

4.2.3.1 Síntesis explicativa del flujo predictivo

En términos simples, PlayerNet es una red neuronal propia del proyecto que recibe 68 variables del jugador y devuelve una probabilidad cruda entre 0 y 1. Esa salida expresa el grado en que el caso se parece a los ejemplos positivos aprendidos durante el entrenamiento; no equivale, por sí sola, a una certeza deportiva.

La probabilidad calibrada es un reajuste posterior de la salida cruda mediante regresión isotónica. Su objetivo es mejorar la interpretación numérica de la probabilidad, no garantizar mejores valores en todas las métricas. Por eso una versión calibrada puede ser más prudente y, al mismo tiempo, obtener un ROC-AUC, PR-AUC o F1 levemente inferior.

La salida sigmoid cruda de PlayerNet es una probabilidad sin calibrar. La calibración isotónica produce otra probabilidad y utiliza un umbral de clasificación seleccionado en validation (0,25; 0,825 para la salida cruda). La interfaz presenta además un score combinado con información histórica y posicional; sus bandas visuales son bajo <0,60, medio 0,60–0,79 y alto ≥0,80. Esos límites visuales no son los umbrales usados para medir F1 en test.

La regresión logística cumple el papel de modelo de referencia. En la corrida documentada, su desempeño fue muy similar al de PlayerNet: obtuvo valores levemente superiores en ROC-AUC y F1, mientras que PlayerNet crudo alcanzó el mayor PR-AUC. En consecuencia, el trabajo

no sostiene una superioridad global de la red neuronal; la conclusión es que una arquitectura más compleja no garantiza mejores resultados y debe seleccionarse según evidencia.

4.2.4 Seguridad y autenticación

La autenticación se basa en sesiones, contraseñas hasheadas, protección CSRF y autorización por rol. Las medidas se contrastaron con riesgos de control de acceso, fallos criptográficos e inyección descritos por OWASP (2021), sin presentar esa referencia como una certificación de seguridad.

El login aplica rate limiting local por nombre de usuario normalizado y no utiliza encabezados de IP controlables por el cliente para construir la clave. Esta mitigación fue verificada para una instancia, pero no sustituye un limitador distribuido en un despliegue multi-instancia.

4.3 Requisitos del sistema

ID	Requisito	Estado
RF-01	Autenticación por sesión para acceso a la aplicación.	Implementado
RF-02	Gestión de usuarios con roles administrador, scout y director.	Implementado
RF-03	Alta, edición y baja de jugadores.	Implementado
RF-04	Ficha de jugador con perfil, edad, categoría, atributos, historial y predicción.	Implementado
RF-05	Registro de atributos en escala 1-20 con historial.	Implementado
RF-06	Registro de rendimiento, partidos, disponibilidad, físico y reportes.	Implementado
RF-07	Cálculo de potencial con PlayerNet en PyTorch.	Implementado

ID	Requisito	Estado
RF-08	Clasificación de potencial: bajo <60%, medio 60-79%, alto >=80%.	Implementado
RF-09	Carga masiva CSV compatible con Excel y validación por fila.	Implementado
RF-10	Panel general con métricas accionables por rol.	Implementado
RF-11	Comparadores 1v1 y múltiple.	Implementado
RF-12	Healthcheck y deploy en Render.	Implementado

TABLA 4-4. REQUISITOS FUNCIONALES DEL SISTEMA.

4.4 Modelo de datos

El modelo de datos se implementa con SQLAlchemy. La entidad central es Player, vinculada a historiales de atributos, estadísticas, partidos, disponibilidad, evaluaciones físicas y reportes scout.

Entidad	Propósito
Player	Jugador, datos personales, posición, club, atributos base, birth_date, photo_url y potencial.
Match	Partido y contexto competitivo.
Coach	Entrenador registrado.
Director	Directivo registrado.
User	Cuenta, credenciales y rol de acceso.
PlayerStat	Estadísticas de rendimiento por fecha.
PlayerAttributeHistory	Historial de atributos técnicos, físicos y mentales.

Entidad	Propósito
PlayerMatchParticipation	Participación del jugador en un partido.
ScoutReport	Reporte cualitativo de scouting.
PhysicalAssessment	Evaluación física y mediciones.
PlayerAvailability	Disponibilidad, fatiga y estado físico.

TABLA 4-5. ENTIDADES PRINCIPALES DEL MODELO DE DATOS.

La Figura 4-2 muestra las clases principales del modelo de dominio persistido con SQLAlchemy y sus relaciones más relevantes.

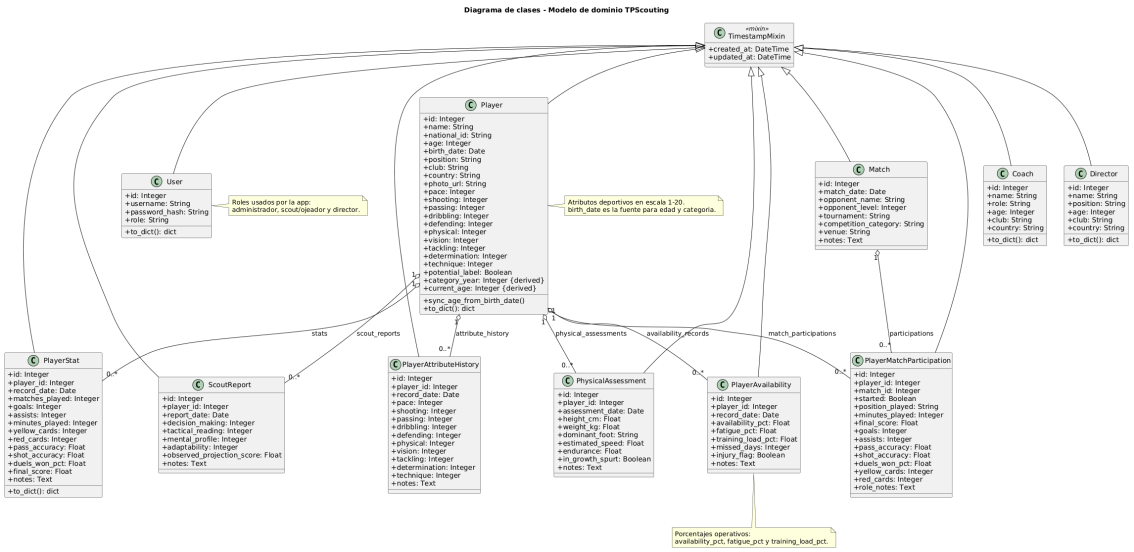


FIGURA 4-2. DIAGRAMA DE CLASES DEL MODELO DE DOMINIO.

4.5 Endpoints principales del MVP

Método	**Ruta**	**Propósito**
GET/POST	/login	Autenticación.
POST	/logout	Cierre de sesión con CSRF.
GET/POST	/register	Alta de usuarios según permisos.
GET	/dashboard	Panel general dinámico por rol.
GET	/players	Listado paginado y filtrable de jugadores.

Método	Ruta	Propósito
GET/POST	/players/manage	Alta individual de jugador.
GET/POST	/players/import	Carga masiva CSV con previsualización.
GET/POST	/player/<id>/stats	Visualización y carga de rendimiento.
GET/POST	/player/<id>/attributes	Visualización y carga de atributos.
GET	/player/<id>/predict	Vista de proyección de potencial.
GET/POST	/compare	Comparador 1v1.
GET/POST	/compare/multi	Comparador múltiple.
GET/POST	/settings	Operaciones administrativas y pipeline.
GET	/health	Estado de servicio, base y calidad de datos.

TABLA 4-6. ENDPOINTS PRINCIPALES DEL MVP.

4.6 Decisiones tecnológicas y alineación con el MVP

Tecnología	Justificación
Flask	Permite desarrollo web server-side simple, auditable y suficiente para un MVP académico.
SQLAlchemy	Abstrae SQLite local y PostgreSQL en Render sin reescribir las reglas del dominio.
PyTorch	Permite implementar PlayerNet y guardar checkpoints con metadatos de entrada.
Scikit-learn	Aporta métricas, partición, modelo de referencia y calibración.
Bootstrap/Jinja2	Facilita una interfaz funcional y adaptable sin incorporar una SPA compleja.
Chart.js	Permite integrar visualizaciones ligeras en las vistas HTML.
pytest/pytest-cov	Proporciona validación automatizada y medición de cobertura en integración continua.
Render/Gunicorn	Permite un despliegue público para validación académica mediante HTTPS.

TABLA 4-7. DECISIONES TECNOLÓGICAS.

4.7 Casos de uso

ID	Caso de uso	Precondición	Resultado
CU-01	Iniciar sesión	Usuario registrado	Sesión activa y acceso según rol.
CU-02	Registrar jugador	Scout o administrador	Jugador persistido con fecha de nacimiento, edad y categoría.
CU-03	Cargar atributos	Jugador existente	Historial actualizado en escala 1-20.
CU-04	Cargar rendimiento/reportes	Jugador existente	Datos operativos visibles en ficha y panel general.
CU-05	Calcular potencial	Modelo cargado y jugador con datos	Probabilidad, categoría y sugerencias visibles.
CU-06	Comparar jugadores	Dos o más jugadores seleccionados	Tabla/gráfico comparativo.
CU-07	Revisar panel general	Usuario autenticado	KPIs y listas accionables por rol.

TABLA 4-8. CASOS DE USO PRINCIPALES.

La Figura 4-3 resume los casos de uso principales y agrupa los roles de administrador, scout y director, junto con el sistema de aprendizaje automático.

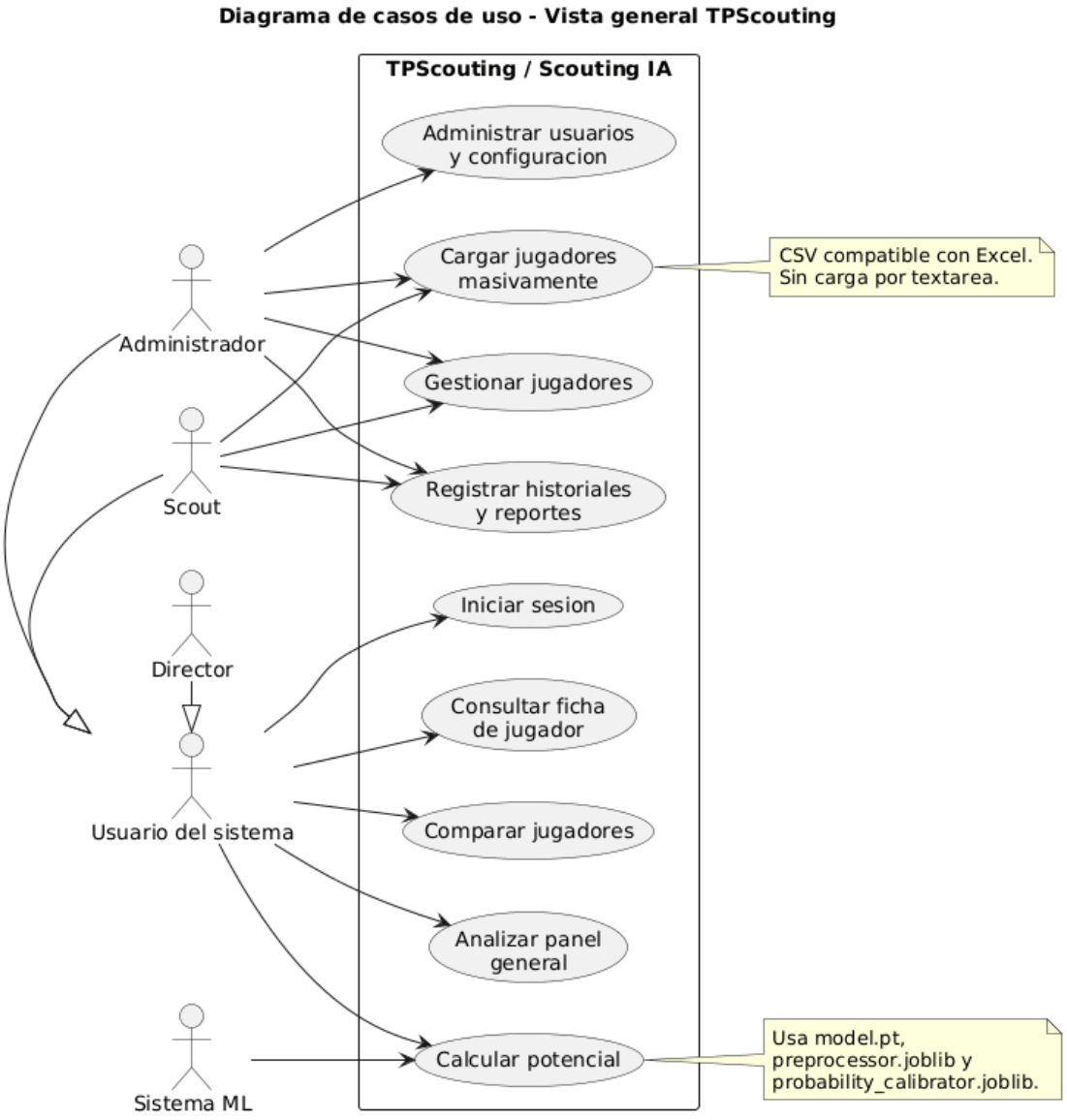


FIGURA 4-3. DIAGRAMA DE CASOS DE USO GENERAL

La Figura 4-4 detalla los casos de uso asociados a alta, edición, consulta, carga masiva e historiales de jugadores.

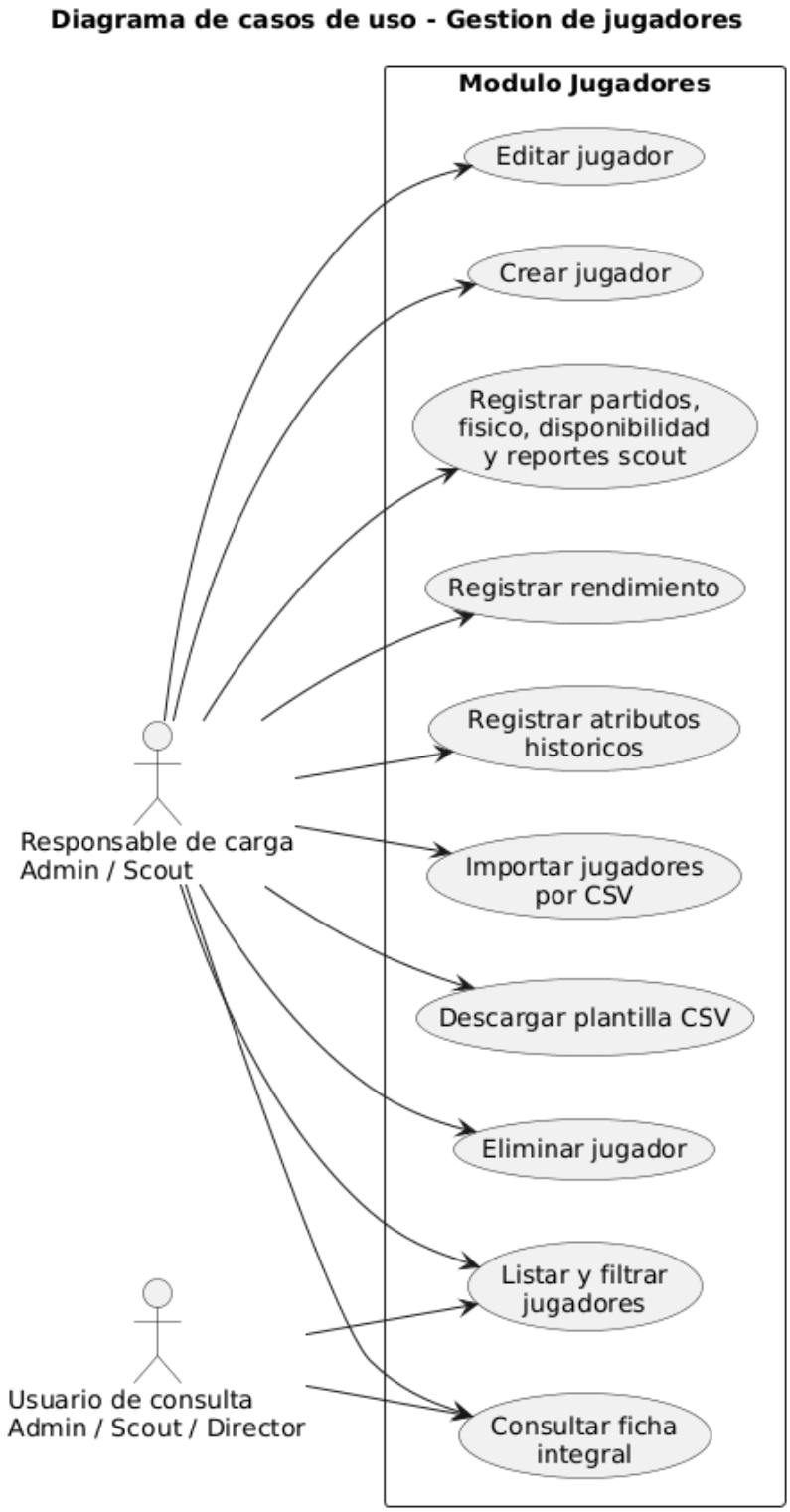


FIGURA 4-4. DIAGRAMA DE CASOS DE USO DE GESTIÓN DE JUGADORES.

La Figura 4-5 muestra los casos de uso orientados a consulta, comparación, panel general y predicción de potencial.

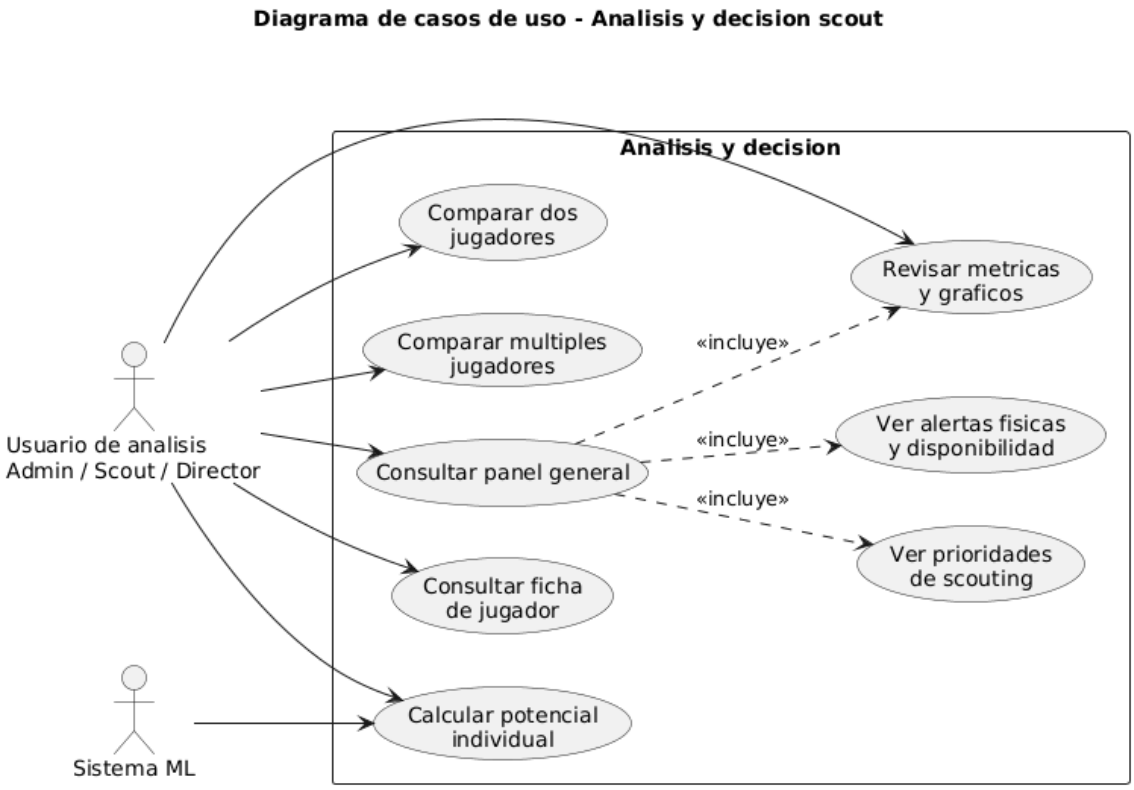


FIGURA 4-5. DIAGRAMA DE CASOS DE USO DE ANÁLISIS Y DECISIÓN SCOUT.

5. DESARROLLO

5.1 Implementación del backend

El backend se implementó en Flask con rutas server-side y blueprints por familia funcional. app.py conserva la inicialización, configuración, registro de blueprints, healthcheck, carga de artefactos ML y compatibilidad con endpoints históricos. Las rutas específicas se distribuyen en scouting_app/routes/auth.py, staff.py, players.py, dashboard.py, compare.py y settings.py.

Se incorporaron servicios auxiliares para seguridad, cache, locks y datos operativos. Esta separación mejora la mantenibilidad sin convertir el MVP en una arquitectura de microservicios innecesaria para el alcance del trabajo.

La integración con PyTorch se mantiene dentro del backend Flask. Los artefactos model.pt, preprocessor.joblib y probability_calibrator.joblib se cargan al iniciar la aplicación y se validan contra input_dim para evitar incompatibilidades entre entrenamiento e inferencia.

Flujo técnico de inferencia: Flask recupera el jugador y sus historiales mediante SQLAlchemy; el preprocesador transforma las 64 columnas de entrada en un vector de 68 variables; PlayerNet produce logits y sigmoid obtiene base_prob, la probabilidad cruda. Si está disponible, el calibrador isotónico genera una referencia secundaria. La interfaz calcula combined_prob mediante combine_probability(base_prob, stats_summary, fit_score), combinando la señal cruda, el promedio histórico y el ajuste posicional, con renormalización cuando falta algún componente. combined_prob es el valor principal mostrado por la

interfaz, pero se interpreta como una heurística operativa y no como una probabilidad calibrada ni como una mejora predictiva demostrada.

La Figura 5-1 detalla el flujo real de inferencia y separa las ramas de error: si el modelo no está disponible, la ruta responde HTTP 500; si el modelo está cargado, pero no existen datos suficientes, se renderiza una vista controlada sin proyección.

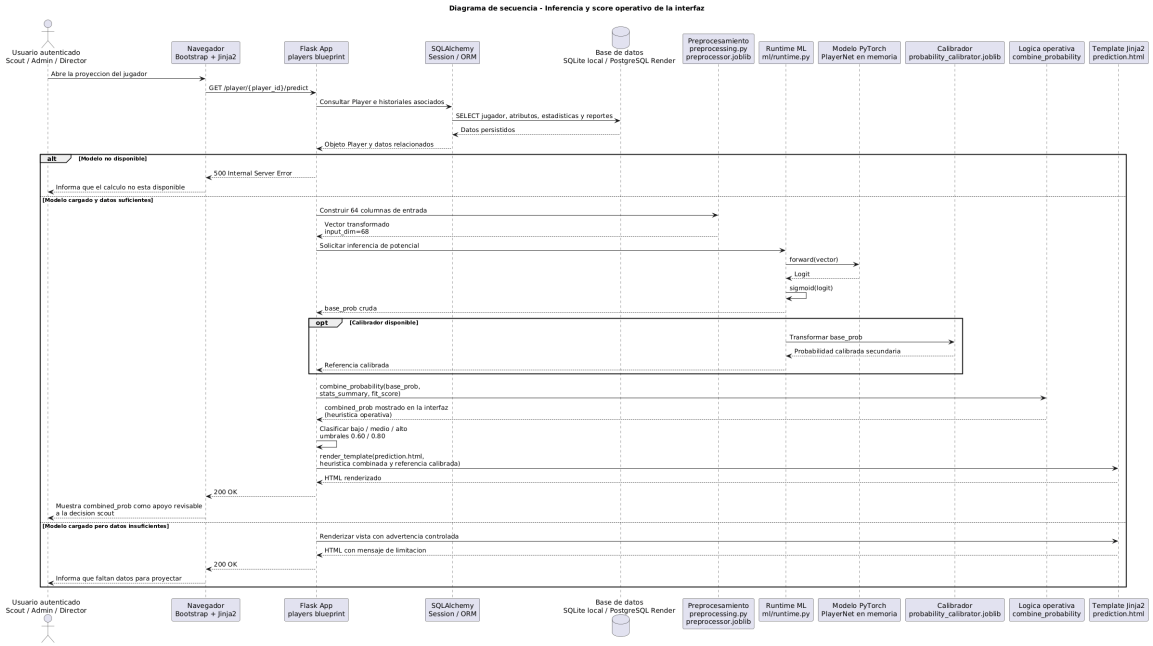


FIGURA 5-1. SECUENCIA DE INFERENCIA: SCORE OPERATIVO DE INTERFAZ Y REFERENCIA CALIBRADA.

5.2 Implementación del frontend

El frontend utiliza Jinja2, Bootstrap, CSS propio y Chart.js. La UX fue ajustada para panel general, fichas de jugador, comparadores, formularios y modales CRUD. La carga de registros históricos se separó visualmente de la lectura de datos mediante modales/offcanvas, evitando formularios extensos incrustados en la pantalla principal.

La Figura 5-2 describe la carga del panel general, incluyendo validación de sesión, caché en memoria, consultas SQLAlchemy y renderizado de la vista.

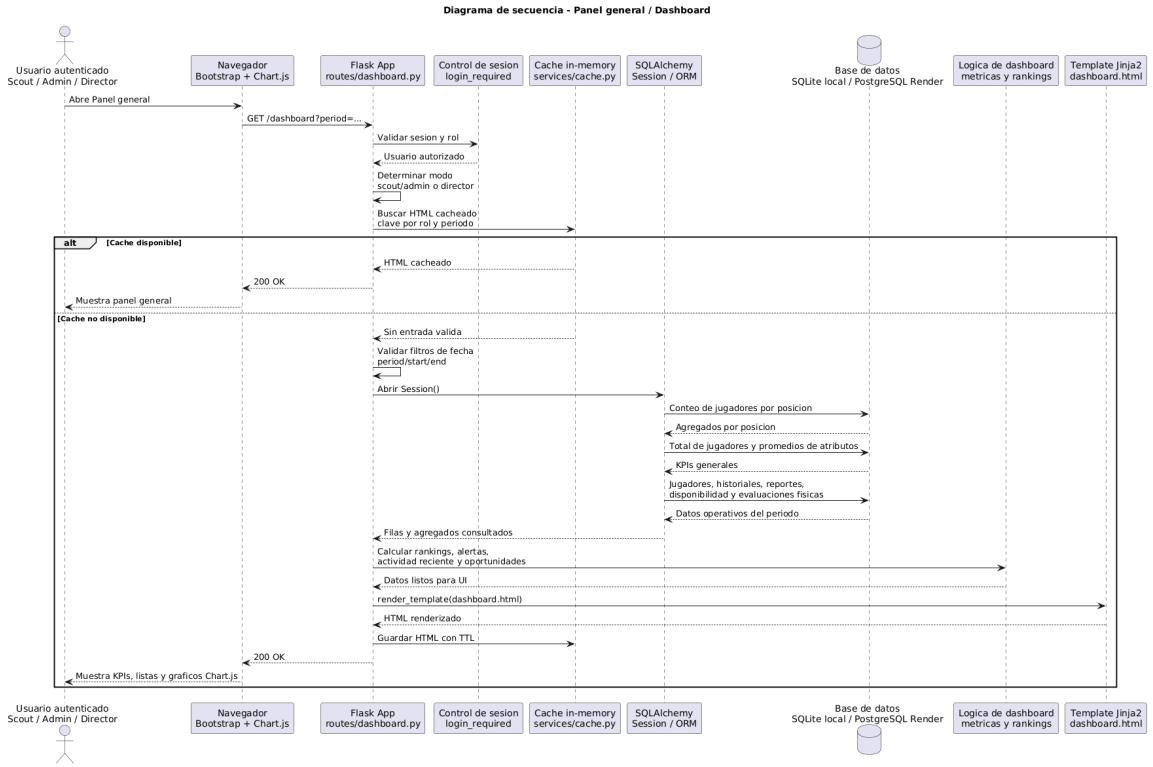


FIGURA 5-2. SECUENCIA DE CARGA DEL PANEL GENERAL.

5.3 Datos, edad y categoría juvenil

La fecha de nacimiento se consolidó como fuente de verdad. A partir de birth_date se calcula la edad actual y la categoría juvenil visible como “Cat. YYYY”. Para datos heredados sin fecha, se generaron fechas determinísticas que conservan edades entre 12 y 18 años, dejando documentado que son datos sintéticos.

5.4 Entrenamiento del modelo de aprendizaje automático

PlayerNet contiene 17.608 parámetros entrenables contados con model.parameters(). Su state_dict contiene 17.994 elementos porque

suma 386 buffers de BatchNorm. La arquitectura combina una rama lineal 68→1 con una rama residual 68→128→64→1, BatchNorm, GELU y dropout 0,15. El entrenamiento utiliza BCEWithLogitsLoss, AdamW y learning rate inicial 0,0005.

Elemento	**Valor verificado**
Clase	PlayerNet
Variable objetivo	temporal_target_label
Dataset y partición	20.000 sintéticos: 14.000 train, 3.000 validation y 3.000 test
Distribución positiva	1.597/20.000 (7,985 %); test 240/3.000 (8 %)
input_dim	68 = 11 variables base + 52 históricas + 5 de posición
Arquitectura	Rama lineal 68->1 + rama residual 68->128->64->1; BatchNorm, GELU y dropout 0,15
Parámetros entrenables	17.608 (model.parameters()); 386 buffers; 17.994 elementos totales en state_dict
Loss	BCEWithLogitsLoss con pos_weight=11,5224
Optimizador	AdamW; weight decay 0,0005
Learning rate / batch size	0,0005 / 256
Épocas solicitadas / ejecutadas	45 / 15
Mejor época / early stopping	Época 5; patience=10; monitor PR-AUC calibrado de validation y F1 como desempate
Training loss	0,6431 en mejor época; 0,5179 en última época
Validation loss / duración	No registradas en la corrida
Calibración	Regresión isotónica; threshold 0,25

TABLA 5-1. CONFIGURACIÓN VERIFICADA DEL MODELO PLAYERNET.

La selección del modelo no se presenta como una superioridad automática del aprendizaje profundo. La regresión logística balanceada y PlayerNet obtuvieron desempeños similares, con ventajas diferentes según la métrica. Por eso la regresión logística se conserva como

modelo de referencia obligatorio y PlayerNet como modelo operativo integrado al MVP.

El pipeline registra metadata junto con el modelo: versión del checkpoint, input_dim, semilla, partición, historial de training loss, métricas de validación y test, baselines y artefactos generados. Se solicitaron 45 épocas; la mejor validación se alcanzó en la época 5 y el entrenamiento se detuvo en la época 15 después de diez épocas sin mejora, de acuerdo con patience=10. La corrida no conserva validation loss, duración total ni el SHA Git exacto del entrenamiento, por lo que esos datos se declaran como no disponibles.

5.5 Seguridad y robustez implementada

• APP_SECRET_KEY obligatoria en producción y limpieza de sesión al autenticar.

• Protección CSRF con comparación de tiempo constante y formularios POST para operaciones mutantes. Las credenciales incompletas y los tokens CSRF no válidos se rechazan con una respuesta controlada, sin iniciar sesión.

• Redirección next limitada a destinos internos y rechazo de esquemas, barras invertidas y rutas //.

• Revalidación del usuario, estado y rol contra la base en cada solicitud protegida.

• Rate limiting local por nombre de usuario normalizado, sin confiar en encabezados de IP suministrados por el cliente; al utilizar memoria de proceso, no cubre varias instancias.

• /health público sin excepciones internas, CSP compatible con los recursos inventariados y validación central de photo_url. La cabecera HSTS se habilita en modo de producción; su efectividad depende del entorno HTTPS y de la configuración de terminación TLS y del proxy.

• Entrenamiento y generación de datos desactivados en requests de producción; los artefactos joblib y PyTorch deben provenir de fuentes confiables.

5.6 Despliegue del sistema

El MVP se desplegó en Render con Gunicorn y PostgreSQL administrado. Debido a las limitaciones del plan, se utiliza una sola base PostgreSQL para la demostración. La aplicación se valida mediante /health, el inicio de sesión, el panel general, el listado, los comparadores y las vistas de jugador.

Elemento	Estado
URL pública	https://tpscouting-mvp.onrender.com
Servidor WSGI	Gunicorn --workers 1 --threads 2
Base en deploy	PostgreSQL administrado Render Free
Datos demo	100 jugadores con birth_date, national_id y photo_url completos
Modelo	Artefactos versionados: model.pt, preprocessor.joblib, probability_calibrator.joblib
Rama de deploy	Evidencia histórica: render-free-deploy. La rama conectada actualmente en Render no fue verificada.

TABLA 5-2. ESTADO DEL DESPLIEGUE EN RENDER.

En Render se evita SQLite como almacenamiento productivo debido al filesystem efímero. El despliegue utiliza PostgreSQL administrado,

variables de entorno, Gunicorn con un worker y dos threads, y un seed demo idempotente para asegurar que la demo pública arranque con datos verificables.

La Figura 5-3 representa la arquitectura utilizada en el despliegue histórico de Render con Gunicorn, Flask, PostgreSQL y artefactos de aprendizaje automático. El despliegue automático depende de la rama configurada en Render; la auditoría no pudo verificar cuál estaba conectada en ese momento. La caché y la limitación de solicitudes residen en memoria, mientras que el bloqueo del pipeline combina un bloqueo de hilo con un archivo creado de forma atómica.

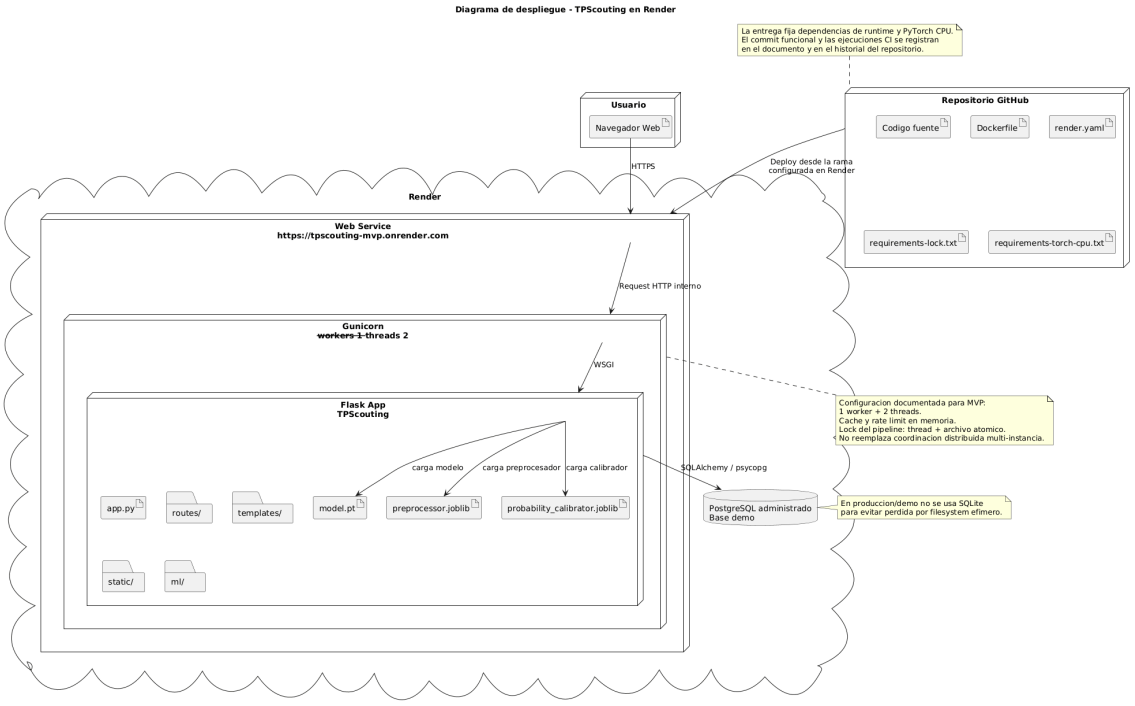


FIGURA 5-3. ARQUITECTURA DE DESPLIEGUE EN RENDER Y MECANISMOS OPERATIVOS DEL MVP.

5.7 Problemas técnicos y decisiones de implementación

Durante el desarrollo ocurrieron algunos problemas que condicionaban la reproducibilidad y el despliegue, y que finalmente se pudieron solucionar. Las principales decisiones tomadas fueron las siguientes:

• Persistencia en Render: se reemplazó SQLite como almacenamiento del despliegue por PostgreSQL administrado, debido al sistema de archivos efímero del servicio.

• Alineación entre entrenamiento e inferencia: se persisten el preprocesador y el checkpoint, y el runtime valida la versión y el input_dim antes de utilizar el modelo.

• Concurrencia del pipeline: se incorporaron bloqueos de hilo y archivo y se configuró un único worker en la demo. Esta solución no equivale a un lock distribuido.

• Rendimiento de la demo: se incorporaron caché con TTL, límite de entradas y paginación. El arranque frío de Render Free continúa siendo una limitación.

6. PRUEBAS Y RESULTADOS

6.1 Plan de pruebas

El plan de pruebas cubre flujos funcionales, seguridad, consistencia de datos, aprendizaje automático, panel general, comparadores, carga masiva, historiales y comportamiento sin modelo cargado. La suite automatizada se ejecuta con pytest; pytest-cov mide cobertura y GitHub Actions prueba Python 3.11 y 3.12.

Tipo	Cobertura	Evidencia
Funcional	Login, logout, rutas protegidas, jugadores, staff, comparadores, panel general.	pytest
Seguridad	CSRF, roles, password mínima, secret obligatoria, rate limiting login.	pytest
Datos	birth_date, categoría, escala 1-20, migraciones manuales y validaciones.	pytest
ML	checkpoint, input_dim, métricas, warnings y compatibilidad legacy.	pytest
UI/Smoke	Render real y Playwright opcional.	Smoke HTTP / capturas

TABLA 6-1. COBERTURA DEL PLAN DE PRUEBAS.

6.2 Resultados del MVP

La verificación local del 20/10/2026 obtuvo 131 pruebas aprobadas, 1 omitida y 4 advertencias conocidas, con una cobertura de 83,31 %. La suite incluye regresiones para credenciales incompletas, tokens CSRF no ASCII y cierre de la sesión de base ante errores de autenticación. La

prueba omitida corresponde al smoke visual Playwright opt-in; la verificación funcional adicional de la demo se realizó con SQLite temporal. Las advertencias corresponden a RuntimeWarning de scikit-learn por columnas completamente NaN en dos pruebas de preprocesamiento. La cobertura global supera el umbral configurado, aunque no es uniforme y no demuestra ausencia de defectos en rutas no cubiertas.

Evidencia	Resultado verificado
pytest local (20/10/2026)	131 passed, 1 skipped, 4 warnings
pytest-cov local	83,31 %
Prueba omitida y demo adicional	Playwright opt-in omitido en la suite; demo funcional adicional verificada con SQLite temporal y 60 jugadores sintéticos
Warnings	4 All-NaN slice encountered en dos pruebas de preprocesamiento
Commit funcional del repositorio principal de respaldo	8809760cd20d804927d3604c160aee2459506b3d
Commit funcional verificado del repositorio de entrega	2a27d53ae4f8a4849bee7d1bea362fd9146886b5

Evidencia	Resultado verificado
CI de la versión funcional corregida	39509728373: aprobada; Python 3.11/3.12, Ruff y auditoría de dependencias
Contenedor de la versión funcional corregida	39509728476: construcción y smoke aprobados
Smoke Render	Verificación histórica del 20/05/2026; no acredita disponibilidad actual.

TABLA 6-2. RESULTADOS DE VALIDACIÓN AUTOMATIZADA Y CI.

6.2.1 Métricas de PlayerNet y calibración

La corrida documentada utiliza la semilla 42, 20.000 jugadores sintéticos y una partición de 14.000/3.000/3.000. La clase positiva representa el 8 % del conjunto de prueba. Las métricas se presentan por separado para la salida sigmoid cruda de PlayerNet y para la salida ajustada mediante calibración isotónica; ninguna corresponde al score combinado mostrado por la aplicación.

Métrica	**Calibrado isotónico**	**Sigmoid crudo**
Threshold	0,250	0,825
Accuracy	0,9303	0,9300
ROC-AUC	0,9174	0,9203
PR-AUC	0,5241	0,5461
F1	0,5282	0,5291
Precision	0,5764	0,5728
Recall	0,4875	0,4917
Matriz de confusión	[[2674, 86], [123, 117]]	[[2672, 88], [122, 118]]

TABLA 6-3. MÉTRICAS DE PLAYERNET CALIBRADO Y CRUDO EN TEST.

La accuracy debe interpretarse junto con el desbalance de clases: predecir siempre la clase negativa alcanzaría 92 %. En la variante calibrada, el recall de 0,4875 significa que se identificaron 117 de 240 positivos sintéticos y se omitieron 123. Por este motivo se priorizan PR-AUC, F1 y el análisis de falsos negativos, especialmente porque el objetivo del sistema es evitar que perfiles potencialmente valiosos pasen inadvertidos.

6.2.2 Comparación con baseline

La comparación incluye PlayerNet crudo, PlayerNet calibrado, regresión logística balanceada y un promedio simple de atributos. La regresión logística supera levemente a PlayerNet calibrado en ROC-AUC, PR-AUC y F1. Frente a PlayerNet crudo, obtiene ROC-AUC y F1 apenas superiores, mientras que PlayerNet crudo alcanza el mayor PR-AUC. Las diferencias son pequeñas y no respaldan una superioridad global de la red neuronal.

Un bootstrap pareado no paramétrico de auditoría (2.000 remuestras, semilla 20261005) comparó PlayerNet crudo con regresión logística sobre el test persistido. La diferencia media crudo menos logística fue −0,00015 en ROC-AUC, con intervalo percentil del 95 % [−0,00269; 0,00247], y 0,00819 en PR-AUC, con intervalo [−0,00251; 0,01954]. Ambos intervalos incluyen cero: esta corrida no aporta evidencia suficiente de superioridad; tampoco demuestra equivalencia. El análisis es retrospectivo, utiliza un único split y no reemplaza una evaluación con varias semillas y datos reales.

Modelo	Accuracy	ROC-AUC	PR-AUC	F1
PlayerNet sigmoid crudo	0,9300	0,9203	0,5461	0,5291
PlayerNet calibrado isotónico	0,9303	0,9174	0,5241	0,5282
LogisticRegression balanced	0,9310	0,9205	0,5378	0,5327
Promedio simple de atributos	0,8960	0,8390	0,3513	0,4201

TABLA 6-4. COMPARACIÓN CON BASELINES.

Como se observa en la Figura 6-1, la curva se construye con la historia real persistida: training loss, PR-AUC de validación y F1 de validación por época. La corrida no registró validation loss, por lo que esa serie no se incorpora ni se estima de manera retrospectiva.

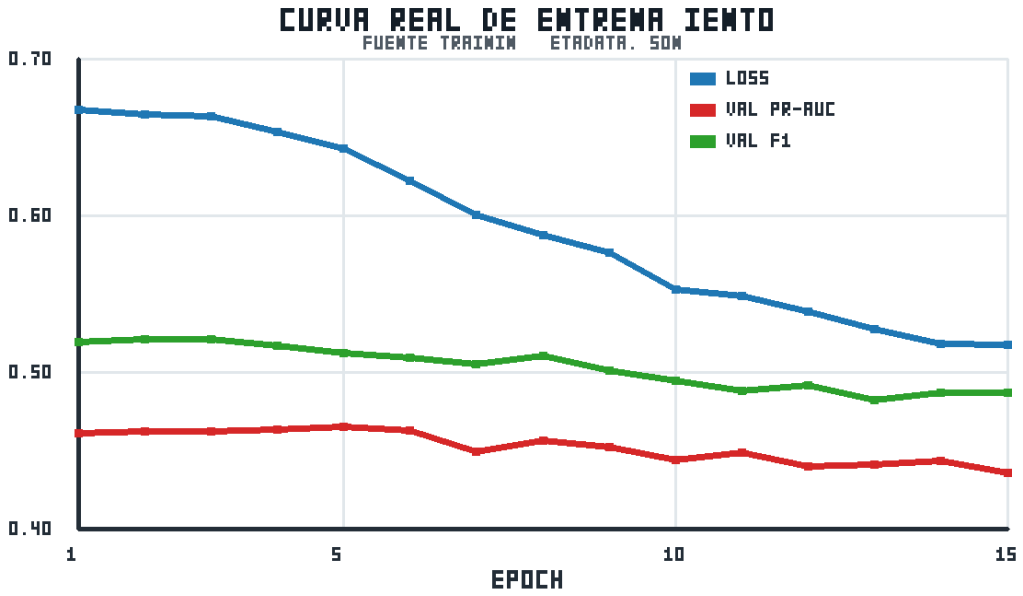


FIGURA 6-1. CURVA REAL DE ENTRENAMIENTO GENERADA DESDE TRAINING_METADATA.JSON.

6.2.3 Evidencia operativa del despliegue

Como evidencia operativa adicional, la Tabla 6-5 presenta el smoke HTTP ejecutado contra Render el 20/05/2026. El primer request refleja

el arranque frío del plan Free; las rutas principales respondieron correctamente en esa verificación histórica. Los controles del 10/07/2026, 13/07/2026 y 21/08/2026 finalizaron por timeout. Por ello no se afirma disponibilidad continua ni estado vigente del servicio; un timeout aislado tampoco demuestra una caída permanente.

Ruta	**Método**	Estado	**Tiempo**	**Resultado**
/health	GET	200	109.36 s	OK; arranque frío Render Free
/login	GET	200	0.39 s	Pantalla de login renderizada
/login	POST	200	4.96 s	Login admin OK; redirección final a /players
/dashboard	GET	200	7.86 s	Panel general renderizado
/players	GET	200	0.38 s	Listado de jugadores
/compare	GET	200	0.43 s	Comparador 1v1
/compare/multi	GET	200	7.73 s	Comparador múltiple
/settings	GET	200	0.44 s	Configuración protegida
/players/import	GET	200	0.39 s	Carga masiva

TABLA 6-5. PRUEBA REAL DE DISPONIBILIDAD DEL DESPLIEGUE EN RENDER.

Las Figuras 6-2 a 6-7 fueron reproducidas el 19 de octubre de 2026 en un navegador real contra una base SQLite temporal con 60 jugadores sintéticos y semilla 42. La sesión autenticada verificó el acceso, el panel general, el listado, la ficha, la predicción y el comparador múltiple. La base temporal se eliminó al finalizar; estas capturas no demuestran disponibilidad continua del despliegue público.

La Figura 6-2 muestra la pantalla de acceso y la diferenciación de roles disponible en el MVP.

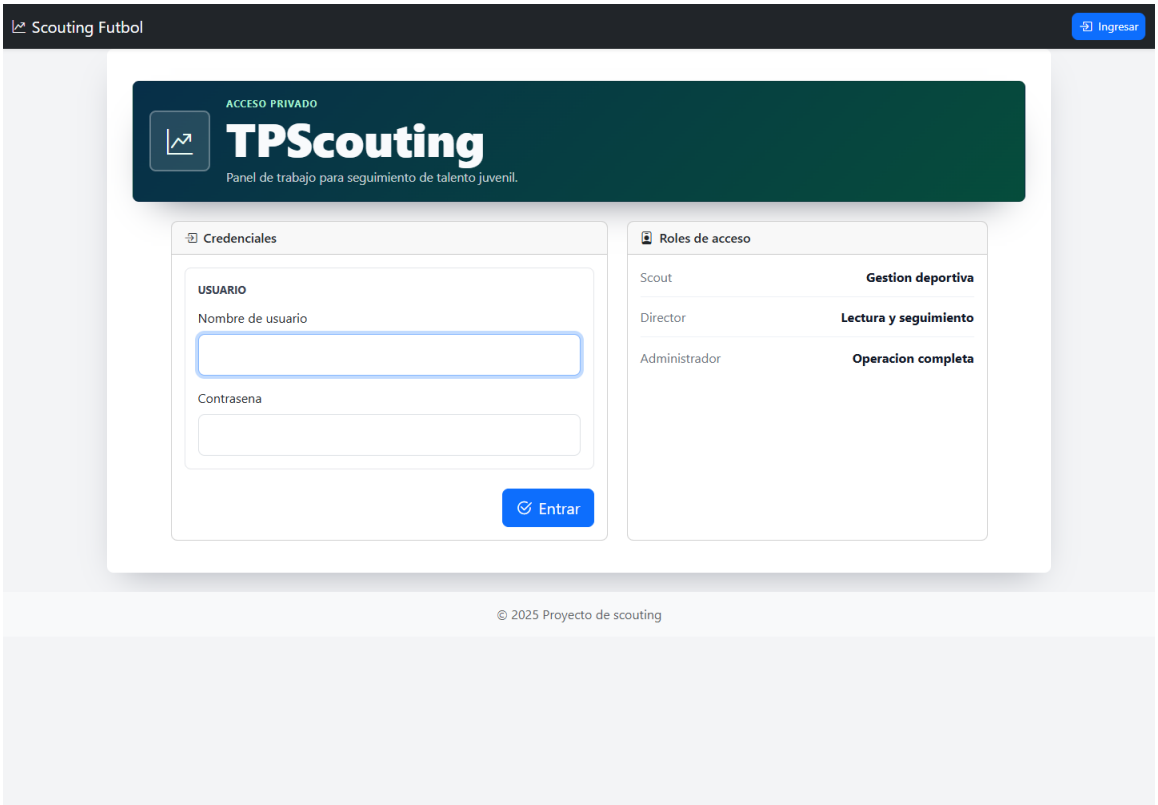


FIGURA 6-2. LOGIN DEL MVP EN LA VERIFICACIÓN LOCAL.

La Figura 6-3 presenta el panel general, que concentra indicadores, prioridades y visualizaciones para apoyar el seguimiento.

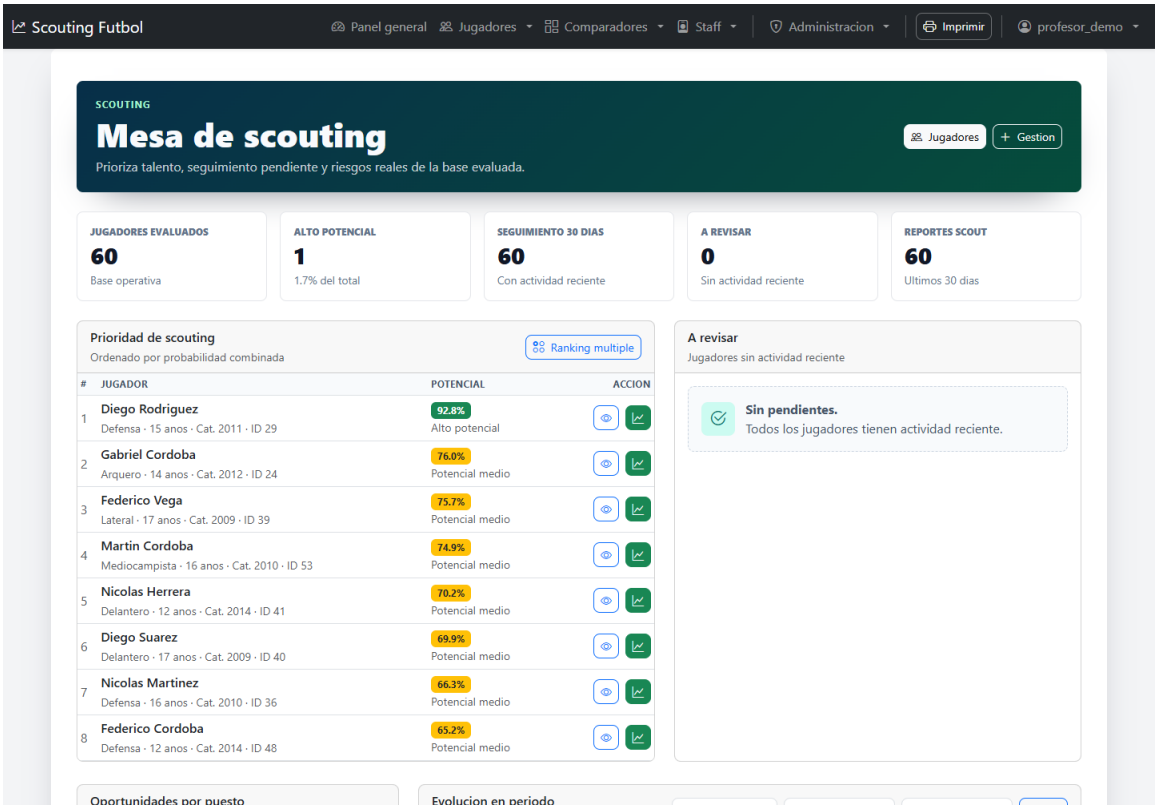


FIGURA 6-3. PANEL GENERAL CON MÉTRICAS POR ROL.

La Figura 6-4 muestra el listado paginado y filtrable de jugadores con edad, categoría y banda de potencial.

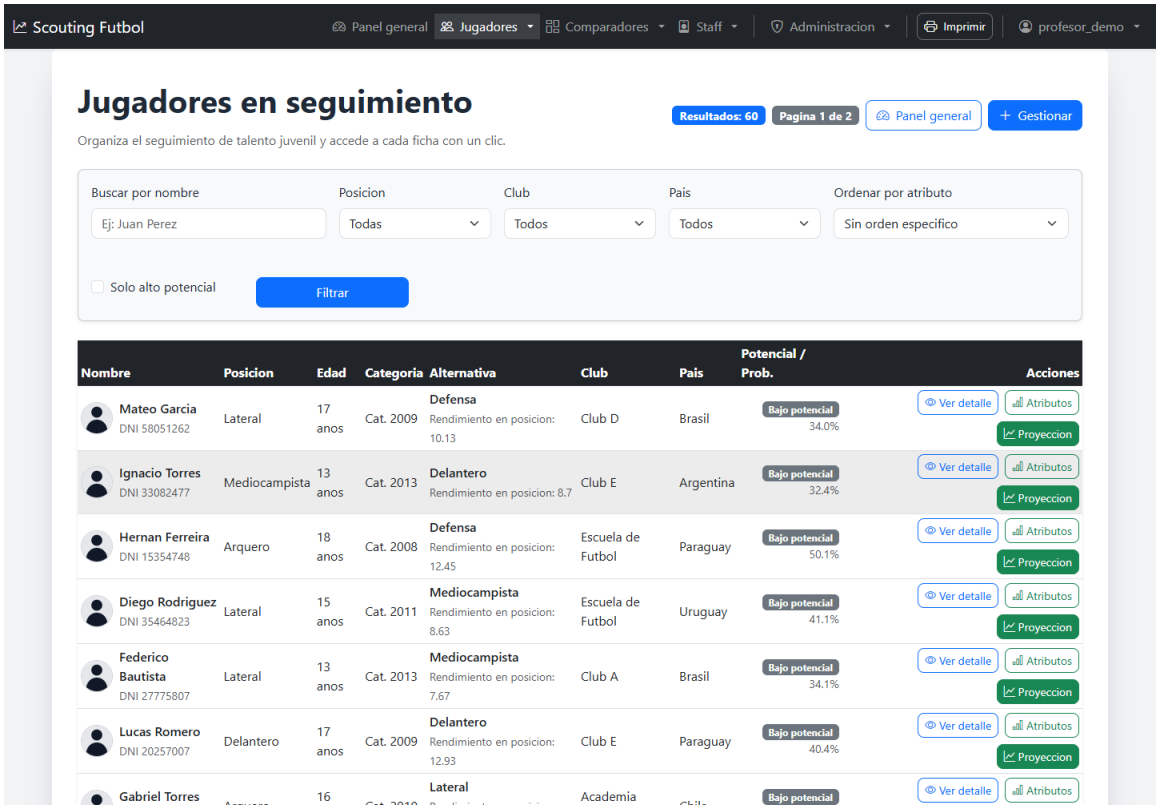
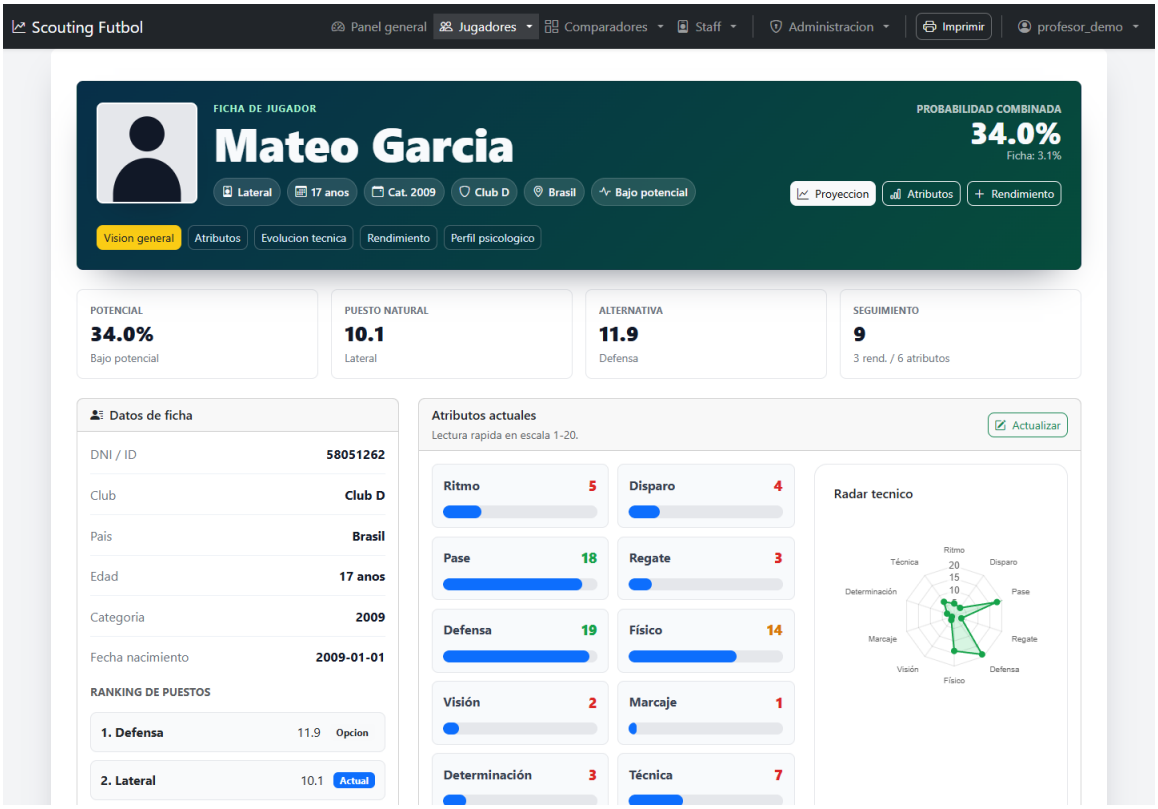
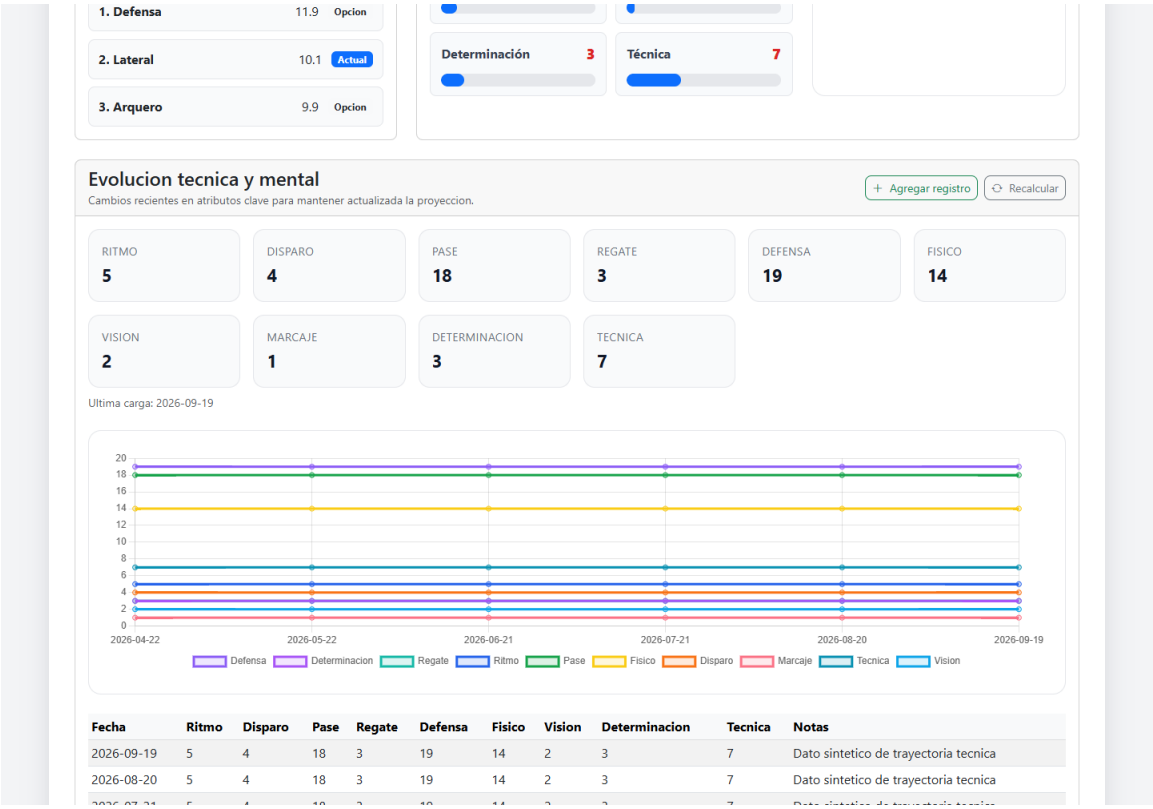


FIGURA 6-4. LISTADO PAGINADO DE JUGADORES CON EDAD, CATEGORÍA Y POTENCIAL.

La Figura 6-5 reúne la ficha integral del jugador, sus atributos, historiales y acciones de mantenimiento.

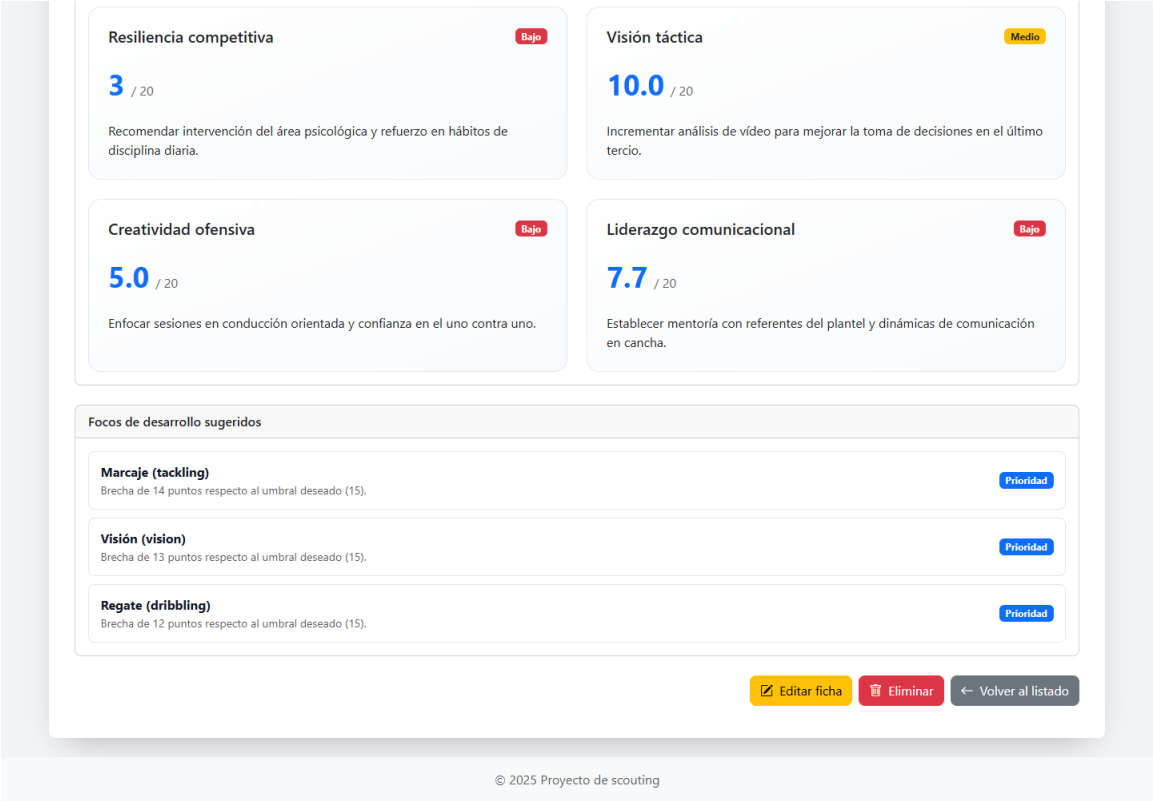


a) Perfil, atributos y resumen de rendimiento.



Fecha	Ritmo	Disparo	Pase	Regate	Defensa	Fisico	Vision	Determinacion	Tecnica	Notas
2026-09-19	5	4	18	3	19	14	2	3	7	Dato sintetico de trayectoria tecnica
2026-08-20	5	4	18	3	19	14	2	3	7	Dato sintetico de trayectoria tecnica

b) Historiales, gráficos y evaluaciones periódicas.



c) Reportes, disponibilidad y acciones de mantenimiento.

FIGURA 6-5. FICHA DE JUGADOR CON HISTORIAL, ATRIBUTOS Y ACCIONES CRUD EN MODALES.

La Figura 6-6 presenta la vista de predicción reproducida en la verificación local del 19 de octubre de 2026, con la edad derivada de

birth_date y el ajuste combinado respecto de PlayerNet crudo. Ese ajuste incorpora el historial disponible y la adecuación posicional.

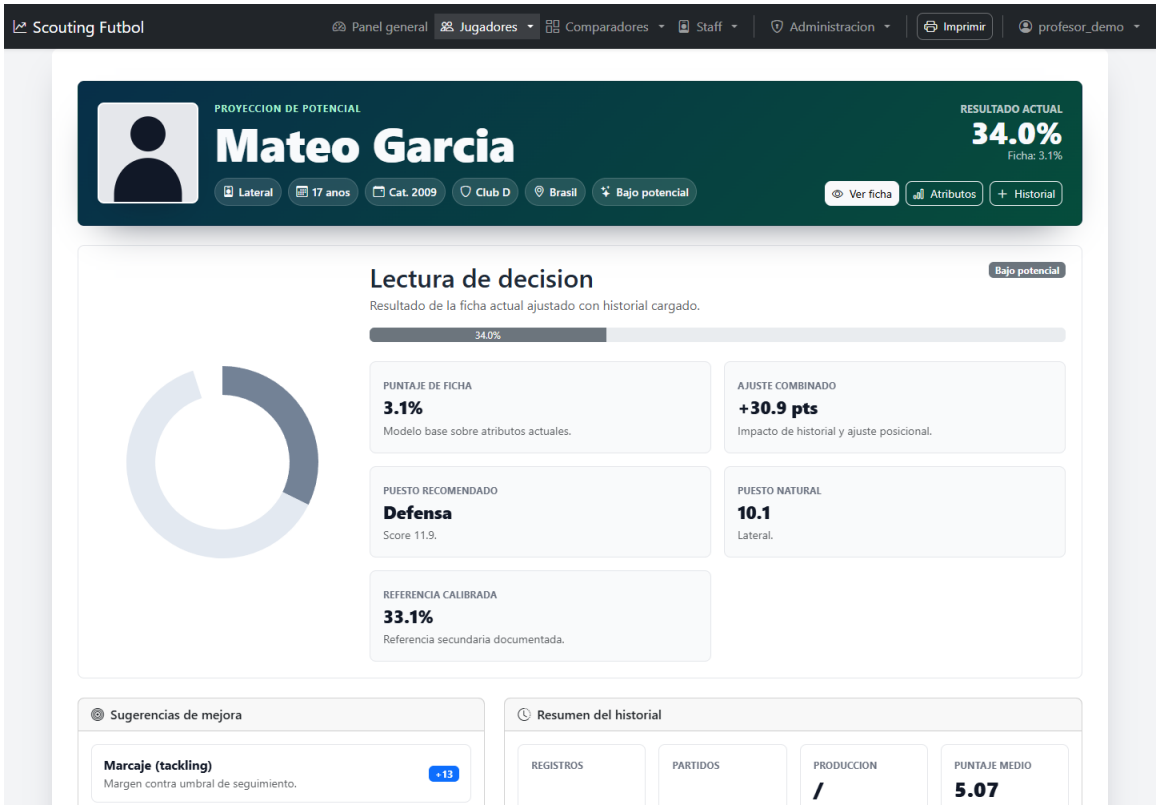


FIGURA 6-6. VISTA DE PREDICCIÓN CON EDAD Y AJUSTE COMBINADO DE HISTORIAL Y POSICIÓN.

La Figura 6-7 muestra el comparador múltiple utilizado para analizar perfiles en forma simultánea.

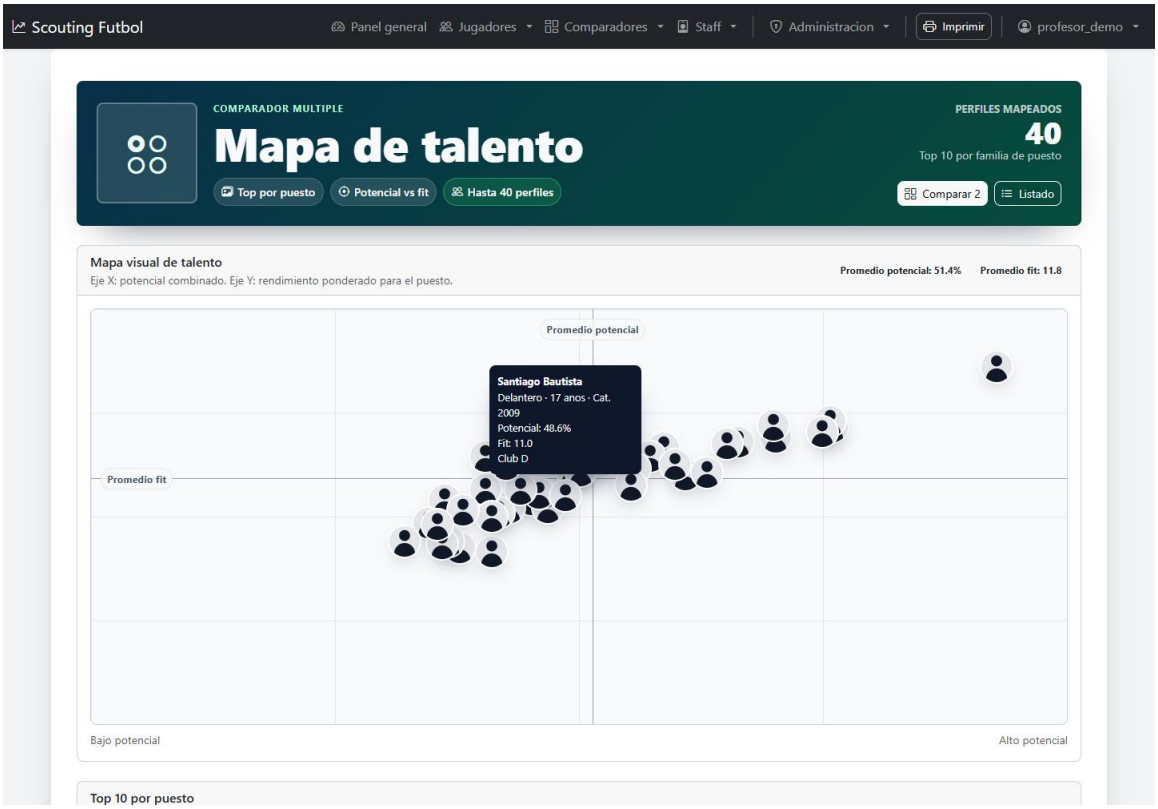


FIGURA 6-7. COMPARADOR MÚLTIPLE DE JUGADORES.

6.2.4 Evidencia de integración continua

La CI #73 se conserva como antecedente histórico del repositorio de desarrollo. La versión funcional corregida del repositorio de entrega corresponde al commit 2a27d53ae4f8a4849bee7d1bea362fd9146886b5, verificado por la ejecución CI 39509728373 y el smoke de contenedor 39509728476, ambos aprobados. La CI incluye Python 3.11 y 3.12, Ruff y auditoría de dependencias. La Figura 6-8 muestra la ejecución anterior 38815440730 del paquete 4c5ef2c y se conserva como evidencia fechada; no sustituye la validación de la versión corregida.

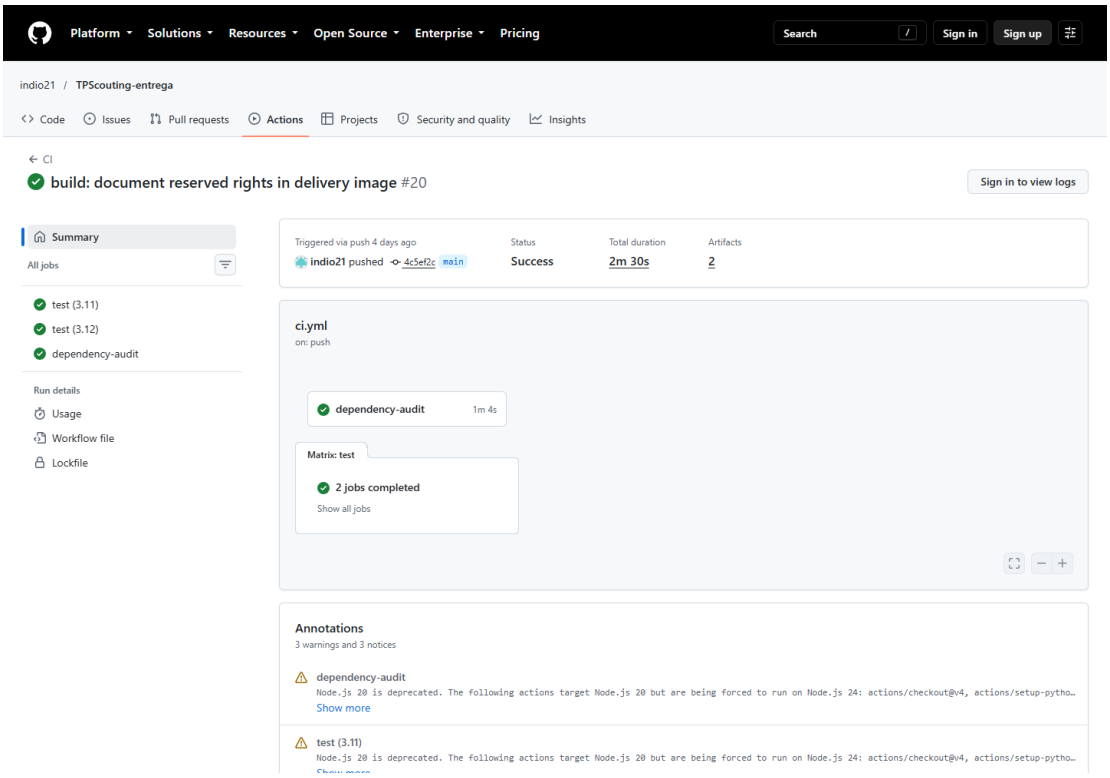


FIGURA 6-8. EJECUCIÓN CI 38815440730 DEL PAQUETE PUBLICADO EN EL REPOSITORIO DE ENTREGA.

6.3 Limitaciones observadas en la validación

• Los datos y los outcomes son sintéticos; las métricas no representan desempeño real en un club.

• La corrida histórica definió cuantiles y cuotas sobre el conjunto completo antes del split. Esta limitación no puede corregirse retrospectivamente; una corrida candidata posterior separó los datos antes de ajustar la política del target y mantuvo los artefactos históricos intactos.

• El score combinado mostrado por la interfaz no formó parte de la evaluación histórica original. Una evaluación posterior sobre test no demostró una mejora defendible frente a la probabilidad cruda y obtuvo peores resultados de Brier y log loss. Por ello se conserva como valor

operativo principal de la interfaz, pero como heurística secundaria desde el punto de vista de la evidencia predictiva; no se presenta como probabilidad ni como mejora demostrada.

• La corrida histórica no registra validation loss, duración total ni SHA Git exacto. Esos datos son irrecuperables y no se reconstruyen. La corrida candidata posterior registra configuración, dependencias, seed, splits, métricas, tiempos y hashes para mejorar la trazabilidad futura.

• La clase positiva representa cerca del 8 %; el recall calibrado de 0,4875 implica una proporción relevante de falsos negativos sintéticos.

• Render Free puede presentar arranque frío, timeout o indisponibilidad temporal; la evidencia operativa debe citarse con fecha.

• La caché, el rate limiting y los locks son locales al proceso; una operación productiva multi-instancia requeriría servicios distribuidos.

6.4 Discusión

Los resultados muestran que el flujo técnico es reproducible en el entorno evaluado, pero no permiten inferir utilidad deportiva. PlayerNet y la regresión logística presentan desempeños próximos en un único split sintético; el intervalo bootstrap de sus diferencias incluye cero. En consecuencia, la complejidad de PlayerNet no queda justificada por una superioridad estadística en esta corrida. La evaluación posterior del score combinado tampoco demostró una mejora defendible frente a la probabilidad cruda. El score se mantiene como valor operativo de la interfaz, pero su interpretación metodológica es la de una heurística secundaria sin superioridad predictiva demostrada.

La comparación entre la corrida histórica y la candidata se limita a la salida cruda de PlayerNet sobre sus respectivos conjuntos de test. La corrida histórica obtuvo ROC-AUC 0,9203, PR-AUC 0,5461, Brier 0,1049, log loss 0,3294 y F1 0,5291. La corrida candidata 20261010T_local_seed42_leakage_safe_v2 obtuvo ROC-AUC 0,9056, PR-AUC 0,4888, Brier 0,1175, log loss 0,3673 y F1 0,4931. En términos descriptivos, la corrida histórica presenta mejores valores en esas cinco métricas.

La diferencia no permite atribuir una pérdida de desempeño únicamente al modelo, porque las corridas no comparten exactamente la misma definición del target. La histórica calculó cuantiles y cuotas antes del split; la candidata ajustó la política solo con train y la aplicó sin recalcularla en validation y test. La candidata aporta un procedimiento metodológicamente más seguro y evidencia reproducible, pero no demuestra una mejora predictiva ni justifica reemplazar los artefactos utilizados por la aplicación. Por ello se conserva como comparación experimental y no como nuevo resultado oficial del MVP.

La validez de la corrida histórica está limitada por una prevalencia fijada por diseño, cuantiles calculados antes del split, una sola semilla y ausencia de jugadores reales. La corrida candidata corrige el ajuste de cuantiles y registra los metadatos faltantes, aunque conserva las limitaciones de una sola semilla y datos sintéticos. El MVP sí aporta evidencia de integración, persistencia, control de acceso y presentación de resultados. Una validación externa requeriría datos

longitudinales reales, consentimiento y criterios definidos antes del experimento.

6.5 Consideraciones éticas y protección de datos

La aplicación fue verificada con datos sintéticos y no se utilizó con menores reales. Un uso real debería definir una base jurídica y una finalidad específica. También debería informar a los titulares y a sus representantes, obtener el consentimiento que corresponda, recolectar sólo los datos necesarios, aplicar plazos de retención y mecanismos de acceso, rectificación y supresión, y limitar el acceso según el rol. La Ley 25.326 regula en Argentina los principios de protección de datos, los derechos de los titulares y las obligaciones de responsables de archivos y bancos de datos (Honorable Congreso de la Nación Argentina, 2000). Estas medidas corresponden a una etapa posterior; el MVP no implementa un flujo completo de consentimiento de tutores ni una política automatizada de retención y baja.

Etiquetar a una persona menor de edad como de «potencial bajo» puede reforzar sesgos de edad relativa, maduración, contexto socioeconómico o calidad del registro, y afectar oportunidades formativas. El score debe tratarse como señal revisable, acompañado por sus límites y nunca como decisión automática. Antes de un piloto real se requieren evaluación ética, análisis de sesgos por subgrupos, supervisión humana, canal de impugnación y seguimiento de consecuencias no deseadas.

7. CONCLUSIONES

7.1 Conclusiones principales

El desarrollo permitió construir un producto mínimo viable que integra el registro de jugadores, el seguimiento de atributos e historiales, la consulta de estadísticas y evaluaciones, la visualización de indicadores, la comparación de perfiles y un módulo de aprendizaje automático. La solución recorre el flujo comprendido entre la carga y persistencia de datos y la presentación de una estimación en la interfaz. Las pruebas automatizadas y las evidencias técnicas demuestran la factibilidad de esa integración dentro del alcance definido.

El aporte comprobado del proyecto reside en centralizar y estructurar información que puede permanecer distribuida entre planillas, observaciones y registros aislados. El sistema permite consultar la evolución de cada jugador y aplicar criterios consistentes de registro. No se midieron reducciones de subjetividad, tiempo de evaluación ni costos operativos; esos beneficios continúan como posibilidades sujetas a validación externa.

El componente de aprendizaje automático integra preprocesamiento, artefactos, calibración e inferencia. Las métricas obtenidas con datos sintéticos permiten describir la corrida y compararla con referencias. La regresión logística obtuvo resultados levemente superiores en algunas métricas, mientras que PlayerNet crudo alcanzó el mayor PR-AUC. El bootstrap no aporta evidencia suficiente de superioridad de ninguno de los dos modelos y tampoco demuestra equivalencia. La

selección futura debe considerar evidencia con varias semillas, costo de errores y condiciones reales de uso.

El objetivo general se alcanzó en su dimensión técnica: se implementó y verificó un MVP que integra registro, análisis, comparación e inferencia. Los objetivos de reducir subjetividad, identificar mejor talento, optimizar recursos o promover igualdad de oportunidades no fueron validados y permanecen como hipótesis de trabajo para estudios con clubes, profesionales y datos reales. El valor comprobado reside en la integración y trazabilidad del flujo, siempre como complemento del juicio humano.

7.2 Cumplimiento de objetivos

Objetivo declarado en 1.2	Estado	Evidencia y alcance
Implementar y verificar el MVP integral	Cumplido localmente	Registro, historiales, paneles, comparadores y PlayerNet; 131 pruebas aprobadas, 1 omitida y 83,31 % de cobertura. Versión funcional de entrega validada por CI 39509728373 y smoke de contenedor 39509728476.

Objetivo declarado en 1.2	Estado	Evidencia y alcance
Estructurar evaluación e historiales	Cumplido a nivel funcional	Escalas, validaciones e historiales implementados; no se midió reducción de subjetividad.
Priorizar perfiles para revisión humana	Implementado, no validado deportivamente	Filtros, comparadores y score disponibles; sin contraste con jugadores o procesos reales.
Apoyar decisiones con información trazable	Cumplido a nivel funcional	Ficha, panel y comparadores integran información; no hubo estudio de uso con scouts.
Evaluar factibilidad para clubes con recursos limitados	Cumplido técnicamente	MVP reproducible; igualdad de oportunidades, adopción y efecto social no medidos.
Organizar y recuperar información	Cumplido a nivel funcional	Persistencia y consulta verificadas; no se cuantificaron

Objetivo declarado en 1.2	Estado	Evidencia y alcance
		tiempos, costos ni ahorro de recursos.

TABLA 7-1. CUMPLIMIENTO DE OBJETIVOS.

7.3 Limitaciones

• Dataset y outcomes completamente sintéticos, sin validación longitudinal con jugadores reales.

• Sin integración de video, tracking, GPS o proveedores comerciales.

• Persistencia y operación suficientes para una demostración, pero no para uso multiusuario intensivo.

• PlayerNet no presenta una superioridad global frente a la regresión logística en la corrida actual.

• La calidad estática se controla mediante un conjunto gradual de reglas Ruff. Permanecen 732 avisos, principalmente vinculados con modernización de anotaciones e importaciones; se consideran deuda técnica controlada y no defectos funcionales demostrados.

• La adopción de una application factory fue evaluada y diferida. El acoplamiento entre inicialización, persistencia, artefactos de aprendizaje automático y pruebas convierte ese cambio en un refactor transversal cuyo riesgo supera el beneficio inmediato para el MVP.

• La compatibilidad fue comprobada en Windows, Linux y macOS sobre Apple Silicon. macOS Intel no forma parte de las plataformas soportadas porque la versión fijada de PyTorch no dispone de una distribución compatible para esa arquitectura.

• El sistema continúa restringido a datos sintéticos. Los controles para consentimiento o representación, ejercicio de derechos, retención, auditoría, seguridad e incidentes fueron diseñados, pero no están implementados ni habilitan el tratamiento de datos reales de menores.

• Los metadatos ausentes de la corrida histórica son irrecuperables. La trazabilidad se corrige hacia adelante mediante una corrida nueva y separada, sin atribuirle retrospectivamente duración, pérdidas o identificadores que no fueron registrados.

7.4 Lecciones aprendidas

El proceso mostró que una arquitectura sencilla, trazable y verificable fue más adecuada para el alcance que una solución distribuida. La comparación experimental confirmó que una red neuronal más compleja no garantiza una mejora frente a un modelo lineal bien configurado.

También se comprobó que la definición del target, la separación temporal de los datos y la trazabilidad de los artefactos son tan importantes como la arquitectura del modelo. Para una siguiente iteración, la prioridad debería ser mejorar el diseño experimental y validar con datos reales antes de aumentar la complejidad algorítmica.

La integración entre desarrollo, pruebas, despliegue y documentación permitió conservar evidencia trazable. El registro de métricas, hashes, cobertura, limitaciones y fechas mejora la reproducibilidad del trabajo.

7.5 Trabajo futuro

• Validar el modelo con datos reales y resultados observables: minutos competitivos, promociones, consenso scout o firma profesional.

• Incorporar auditoría de cambios, registros estructurados, caché distribuida y rate limiting centralizado.

• Separar más lógica de app.py hacia servicios o una app factory si el proyecto crece.

• Agregar exportaciones formales, reportes PDF y flujos de aprobación por rol.

• Evaluar modelos alternativos y seleccionar el más adecuado según evidencia, no por complejidad técnica.

• Reducir gradualmente la deuda estática por regla o módulo, con pruebas específicas y sin aplicar una modernización masiva que dificulte atribuir regresiones.

• Reconsiderar una application factory solo si el crecimiento del proyecto justifica separar el ciclo de vida de base, caché, rate limiting y artefactos, mediante etapas reversibles y pruebas de dos aplicaciones aisladas.

• Evaluar una combinación compatible de Python y PyTorch si se requiere ampliar el soporte a macOS Intel, repitiendo instalación, carga de artefactos, suite y demo antes de declararlo soportado.

• Implementar y validar los controles institucionales, jurídicos y técnicos diseñados antes de cualquier piloto con datos reales de menores, junto con validación externa y supervisión humana.

• Mantener para toda nueva corrida el registro de configuración, dependencias, seeds, splits, métricas, tiempos, commit y hashes, sin modificar ni completar retrospectivamente la evidencia histórica.

8. BIBLIOGRAFÍA

Cobley, S., Baker, J., Wattie, N., & McKenna, J. (2009). Annual age-grouping and athlete development: A meta-analytical review of relative age effects in sport. Sports Medicine, 39(3), 235–256. https://doi.org/10.2165/00007256-200939030-00005

Decroos, T., Bransen, L., Van Haaren, J., & Davis, J. (2019). Actions speak louder than goals: Valuing player actions in soccer. Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 1851–1861. https://doi.org/10.1145/3292500.3330758

Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep learning. MIT Press.

Hastie, T., Tibshirani, R., & Friedman, J. (2009). The elements of statistical learning (2nd ed.). Springer. https://doi.org/10.1007/978-0-387-84858-7

Honorable Congreso de la Nación Argentina. (2000). Ley 25.326 de Protección de los Datos Personales. https://www.argentina.gob.ar/normativa/nacional/ley-25326-2000-64790

Hudl. (s. f.). Wyscout. https://www.hudl.com/products/wyscout

Kaufman, S., Rosset, S., Perlich, C., & Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. ACM Transactions on Knowledge Discovery from Data, 6(4), Article 15. https://doi.org/10.1145/2382577.2382579

Lacan, S. (2024). Stacking-based deep neural network for player scouting in football. arXiv. https://doi.org/10.48550/arXiv.2403.08835

Niculescu-Mizil, A., & Caruana, R. (2005). Predicting good probabilities with supervised learning. Proceedings of the 22nd International Conference on Machine Learning, 625–632. https://doi.org/10.1145/1102351.1102430

OWASP Foundation. (2021). OWASP Top 10: The ten most critical web application security risks. https://owasp.org/Top10/

Pallets Projects. (s. f.). Flask documentation. https://flask.palletsprojects.com/

Pappalardo, L., Cintia, P., Ferragina, P., Massucco, E., Pedreschi, D., & Giannotti, F. (2019). PlayeRank: Data-driven performance evaluation and player ranking in soccer via a machine learning approach. ACM Transactions on Intelligent Systems and Technology, 10(5), Article 59. https://doi.org/10.1145/3343172

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., & Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. Advances in Neural Information Processing Systems, 32. https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830. https://www.jmlr.org/papers/v12/pedregosa11a.html

Pressman, R. S. (2010). Ingeniería del software: Un enfoque práctico (7.ª ed.). McGraw-Hill.

PyTorch Foundation. (s. f.). PyTorch documentation. https://docs.pytorch.org/docs/

Saito, T., & Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. PLOS ONE, 10(3), e0118432. https://doi.org/10.1371/journal.pone.0118432

Schwaber, K., & Sutherland, J. (2020). La Guía Scrum: La guía definitiva de Scrum: Las reglas del juego. https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-Spanish-Latin-South-American.pdf

Sommerville, I. (2011). Ingeniería de software (9.ª ed.). Pearson Educación.

SQLAlchemy. (s. f.). SQLAlchemy documentation. https://docs.sqlalchemy.org/

van Arem, K. W., Goes-Smit, F., & Söhl, J. (2025). Forecasting the future development in quality and value of professional football players. Applied Sciences, 15(16), 8916. https://doi.org/10.3390/app15168916

9. ANEXOS TÉCNICOS

9.1 Rutas principales

Archivo	Rutas relevantes
auth.py	/login, /logout, /register
players.py	/players, /player/<id>, /players/manage, /players/import, historiales y predicción
dashboard.py	/dashboard
compare.py	/compare, /compare/multi
staff.py	/coaches, /directors
settings.py	/settings
app.py	/, /health y registro de blueprints

TABLA 9-1. RUTAS PRINCIPALES.

9.2 Variables de entorno relevantes

Variable	Uso
APP_SECRET_KEY	Obligatoria en producción para firmar sesiones.
APP_DB_URL	Base operativa SQLite local o PostgreSQL en Render.
TRAINING_DB_URL	Base de entrenamiento cuando se ejecuta pipeline.
ADMIN_USERNAME / ADMIN_PASSWORD	Credenciales iniciales del administrador.
ALLOW_DEFAULT_ADMIN	Solo desarrollo local; no usar en producción.
CACHE_TTL_SECONDS / CACHE_MAX_ENTRIES	Cache in-memory configurable.
POTENTIAL_MEDIUM_THRESHOLD / POTENTIAL_HIGH_THRESHOLD	Umbrales documentados; default 0.60 y 0.80.
RENDER_SMOKE_BASE_URL	URL pública usada para smoke externo.

TABLA 9-2. VARIABLES DE ENTORNO RELEVANTES.

9.3 Glosario técnico

Término	Definición
MVP	Producto mínimo viable utilizado para validar el flujo principal del sistema.
PlayerNet	Red neuronal propia del proyecto que recibe 68 variables y produce una probabilidad cruda de pertenencia a la clase positiva sintética.

Término	Definición
Vector de características	Representación numérica de las variables de entrada utilizadas por el modelo.
ROC-AUC	Métrica de separación entre clases para clasificación binaria.
PR-AUC	Métrica útil con clase positiva minoritaria.
BCEWithLogitsLoss	Función de pérdida usada con logits en clasificación binaria.
CSRF	Ataque de falsificación de petición; mitigado con token en formularios POST.
Gunicorn	Servidor WSGI usado para ejecutar Flask en Render.
Render	Proveedor cloud utilizado para la demo pública.
Panel general	Vista de control con indicadores, listas accionables y gráficos para apoyar decisiones.
SQLAlchemy	ORM de Python utilizado para mapear entidades del dominio a tablas relacionales.
PostgreSQL	Motor relacional administrado utilizado en el despliegue público de Render.
SQLite	Motor relacional liviano utilizado para desarrollo local y pruebas del MVP.
Logits	Salida cruda del modelo antes de convertirla a probabilidad mediante sigmoid.
Sigmoid	Función matemática que transforma logits en valores entre 0 y 1.
Baseline	Modelo o regla de referencia usado para comparar el desempeño de una propuesta.
Checkpoint	Archivo que guarda estado del modelo y metadata necesaria para cargarlo de forma compatible.
input_dim	Cantidad de features esperadas por el modelo al momento de inferir.
Smoke test	Prueba rápida de disponibilidad y rutas críticas del sistema desplegado.
CI	Integración continua; ejecución automatizada de pruebas en GitHub Actions.
pytest-cov	Extensión de pytest usada para medir cobertura de pruebas.
temporal_target_label	Variable objetivo binaria sintética utilizada en la corrida oficial; se construye con señales futuras simuladas y reglas temporales.
Probabilidad cruda	Salida sigmoid directa de PlayerNet antes de calibración o combinación con reglas operativas.

Término	Definición
Calibración isotónica	Reajuste no paramétrico de una probabilidad para mejorar su interpretación; no garantiza mejores métricas de ranking.
Score combinado	Valor operativo de la interfaz que integra probabilidad cruda, promedio histórico y ajuste posicional.
Modelo de referencia (baseline)	Modelo simple utilizado como punto de comparación; en este trabajo se usa regresión logística balanceada.
Scouting	En este trabajo, proceso sistemático de observación, registro y comparación de jugadores para apoyar decisiones de identificación y seguimiento deportivo.

TABLA 9-3. GLOSARIO TÉCNICO.

9.4 Reproducción mínima

Repositorio de entrega: https://github.com/indio21/TPScouting-entrega. Versión funcional evaluada: commit 2a27d53ae4f8a4849bee7d1bea362fd9146886b5, CI 39509728373 y smoke de contenedor 39509728476, aprobados. La identificación del paquete documental y los hashes del Word/PDF se registran en VERIFICACION_ENTREGA.md. El repositorio principal conserva el historial de trabajo y las evidencias internas que no forman parte de la entrega académica.

La distribución incluye Dockerfile, .dockerignore y .env.example para reproducir el entorno sin publicar secretos. El repositorio no incluye un archivo LICENSE. Por decisión de su titular, todos los derechos se encuentran reservados y la entrega no concede una autorización general de copia, modificación o redistribución.

Los comandos siguientes se ejecutan desde la raíz del repositorio clonado. En Windows y Linux CPU, el runtime se instala desde requirements-lock.txt y PyTorch desde requirements-torch-cpu.txt; requirements-dev.txt agrega pruebas, auditoría y lint. En macOS se usa requirements.txt para obtener el wheel estándar de PyTorch y luego requirements-dev.txt. requirements-docs.txt se reserva para trabajar con el documento. Se verificó Windows con Python 3.11, Linux con Python 3.11 y 3.12, y macOS Apple Silicon con Python 3.11. macOS Intel no se declara compatible porque PyTorch 2.14.1 no dispone de un wheel x86_64 para esa plataforma.

Crear entorno virtual:

Windows PowerShell: py -3.11 -m venv .venv
Linux/macOS: python3.11 -m venv .venv

Instalar dependencias:

Windows:
.\.venv\Scripts\python.exe -m pip install -r requirements-lock.txt
.\.venv\Scripts\python.exe -m pip install -r requirements-torch-cpu.txt
.\.venv\Scripts\python.exe -m pip install -r requirements-dev.txt

Linux CPU:
.venv/bin/python -m pip install -r requirements-lock.txt
.venv/bin/python -m pip install -r requirements-torch-cpu.txt
.venv/bin/python -m pip install -r requirements-dev.txt

macOS:
.venv/bin/python -m pip install -r requirements.txt
.venv/bin/python -m pip install -r requirements-dev.txt

Crear administrador:

Windows PowerShell:
$env:ADMIN_USERNAME='<usuario-admin>'
$env:ADMIN_PASSWORD='<contraseña-segura>'
.\.venv\Scripts\python.exe .\scouting_app\create_admin.py
Linux/macOS:
ADMIN_USERNAME='<usuario-admin>' ADMIN_PASSWORD='<contraseña-segura>'
.venv/bin/python scouting_app/create_admin.py

Ejecutar la aplicación:

Demo portable recomendada: .\.venv\Scripts\python.exe .\scripts\iniciar_demo.py
Linux/macOS: .venv/bin/python scripts/iniciar_demo.py

Ejecutar pruebas y cobertura:

Windows PowerShell:
.\.venv\Scripts\python.exe -m pytest -q --cov=scouting_app --cov-report=term-missing
Linux/macOS:
.venv/bin/python -m pytest -q --cov=scouting_app --cov-report=term-missing

Pipeline reproducible opcional de datos, entrenamiento, evaluación y sincronización de demo. Debe ejecutarse en una copia de trabajo y con artefactos de salida separados para no sobrescribir la corrida histórica:

Windows PowerShell: Set-Location .\scouting_app
Linux/macOS: cd scouting_app

Generar el dataset sintético oficial:

Windows PowerShell:
..\.venv\Scripts\python.exe generate_data.py --num-players 20000 --db-url sqlite:///players_training.db --seed 42 --min-age 12 --max-age 18 --reset
Linux/macOS:
../.venv/bin/python generate_data.py --num-players 20000 --db-url sqlite:///players_training.db --seed 42 --min-age 12 --max-age 18 --reset

Entrenar PlayerNet con la configuración documentada:

Windows PowerShell:
..\.venv\Scripts\python.exe train_model.py --db-url sqlite:///players_training.db --model-out model_candidate.pt --preprocessor-out preprocessor_candidate.joblib --calibrator-out probability_calibrator_candidate.joblib --metadata-out training_metadata_candidate.json --splits-out training_splits_candidate.json --epochs 45 --lr 5e-4 --patience 10
Linux/macOS:
../.venv/bin/python train_model.py --db-url sqlite:///players_training.db --model-out model_candidate.pt --preprocessor-out preprocessor_candidate.joblib --calibrator-out probability_calibrator_candidate.joblib --metadata-out training_metadata_candidate.json --splits-out training_splits_candidate.json --epochs 45 --lr 5e-4 --patience 10

Evaluar el modelo guardado:

Windows PowerShell:
..\.venv\Scripts\python.exe evaluate_saved_model.py --db-url sqlite:///players_training.db --metadata-path training_metadata_candidate.json
Linux/macOS:
../.venv/bin/python evaluate_saved_model.py --db-url sqlite:///players_training.db --metadata-path training_metadata_candidate.json

Sincronizar la base operativa de demostración:

Windows PowerShell:
..\.venv\Scripts\python.exe sync_shortlist.py --src-db sqlite:///players_training.db --dst-db sqlite:///players_updated_v2.db --limit 100 --min-age 12 --max-age 18 --replace
Linux/macOS:
../.venv/bin/python sync_shortlist.py --src-db sqlite:///players_training.db --dst-db sqlite:///players_updated_v2.db --limit 100 --min-age 12 --max-age 18 --replace

Volver a la raíz del proyecto:

Windows PowerShell: Set-Location ..
Linux/macOS: cd ..

Smoke del despliegue:

Windows PowerShell:
.\.venv\Scripts\python.exe scripts\smoke_render.py --base-url https://tpscouting-mvp.onrender.com
Linux/macOS:
.venv/bin/python scripts/smoke_render.py --base-url https://tpscouting-mvp.onrender.comhttps://tpscouting-mvp.onrender.com

10. ANEXOS VISUALES Y EVIDENCIA

Los anexos visuales complementan la sección de pruebas con capturas del sistema, diagramas ampliados y evidencia de integración continua. Las evidencias se presentan con fecha y alcance para evitar interpretar una verificación histórica como disponibilidad permanente.

10.1 Evidencia de Render

• URL pública validada históricamente el 20/05/2026: https://tpscouting-mvp.onrender.com.

• Login administrativo verificado en el smoke histórico del 20/05/2026.

• Rutas verificadas: /dashboard, /players, /compare, /compare/multi, /settings y /players/import.

• Healthcheck histórico documentado con 100 jugadores demo y faltantes críticos en 0; los controles del 10/07/2026, 13/07/2026 y 21/08/2026 finalizaron por timeout, por lo que no se afirma disponibilidad vigente.

10.2 Integración continua y documentos técnicos

• README.md: instalación, pruebas, despliegue y limitaciones.

• RUNBOOK.md: operación, variables, smoke, Render y riesgos conocidos.

• docs/modelo_ml.md: descripción del modelo, preprocesamiento y artefactos incluidos en la entrega.

• GUIA_DEMO_PROFESOR.md: instrucciones de instalación, ejecución y evaluación de la demo local.

• docs/REAL_DATA_READINESS.md: controles pendientes antes de cualquier uso con datos reales de menores.

La Figura 10-1 presenta una vista ampliada de la ejecución anterior 38815440730 del paquete 4c5ef2c. Las correcciones de autenticación se verificaron posteriormente en el commit funcional 2a27d53ae4f8a4849bee7d1bea362fd9146886b5, mediante la CI 39509728373 y el smoke de contenedor 39509728476, aprobados. La publicación del documento actualizado se identifica por sus hashes y el historial de Git; no se atribuye a una captura histórica la validación de cambios posteriores.

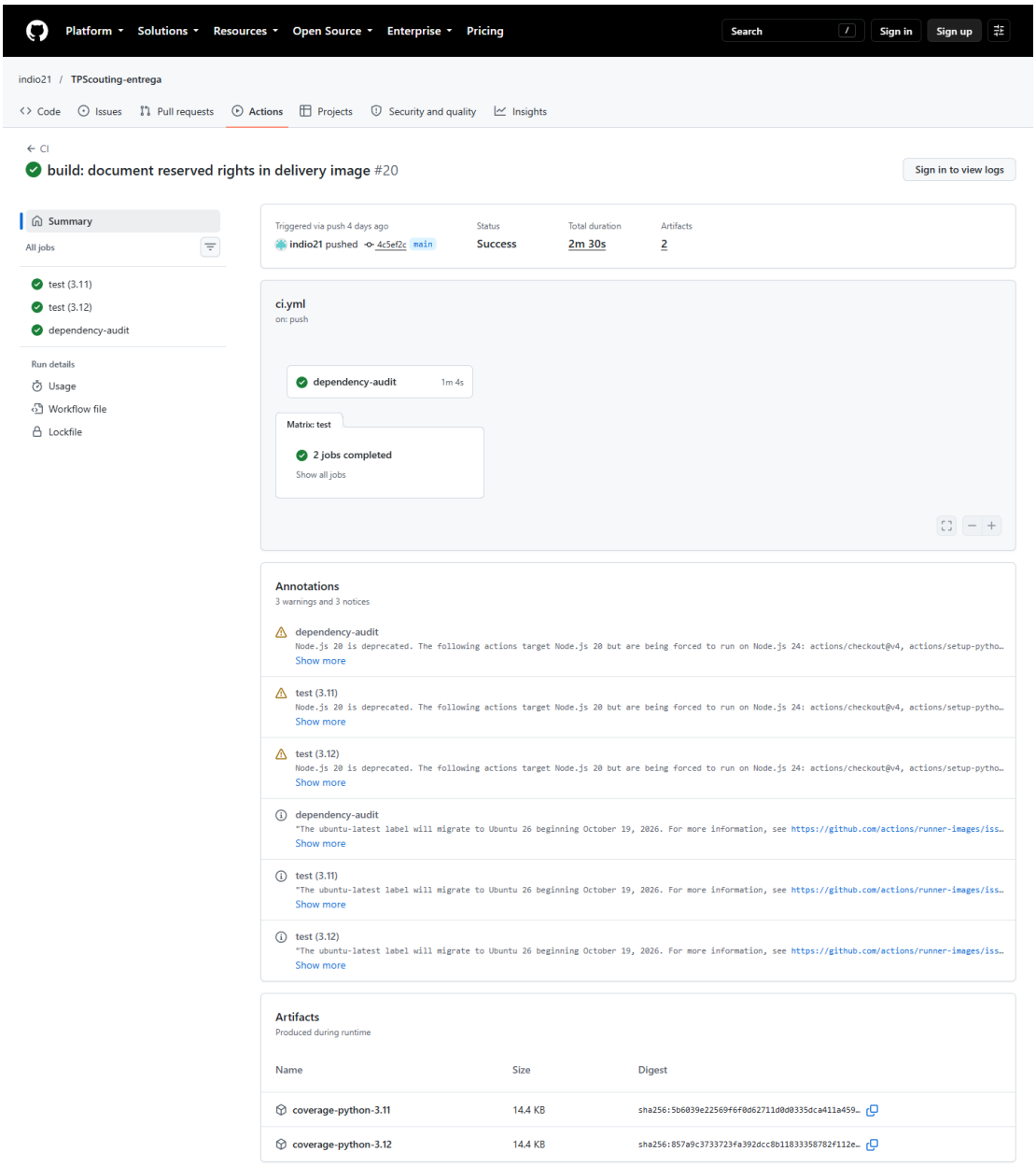


FIGURA 10-1. VISTA AMPLIADA DE LA EJECUCIÓN CI 38815440730 DEL REPOSITORIO DE ENTREGA.

10.3 Limitaciones documentadas del MVP

• Datos y outcomes sintéticos; las métricas no son extrapolables a rendimiento real sin validación externa.

• Render Free puede dormir el servicio y producir latencias iniciales elevadas.

• La app está modularizada por blueprints, pero todavía no es una arquitectura de microservicios.

• La decisión de modelo debe revisarse con datos reales; el baseline logístico fue competitivo en la corrida actual.

10.4 Diagramas ampliados

Esta sección reproduce ocho diagramas del cuerpo en páginas independientes. Se conservan porque el tamaño del cuerpo dificulta leer etiquetas y relaciones; los anexos sirven como ampliación visual y no como evidencia adicional ni como diagramas diferentes.

Las Figuras 10-2 a 10-6 amplían, respectivamente, los componentes, el modelo de dominio, la inferencia, el panel general y el despliegue.

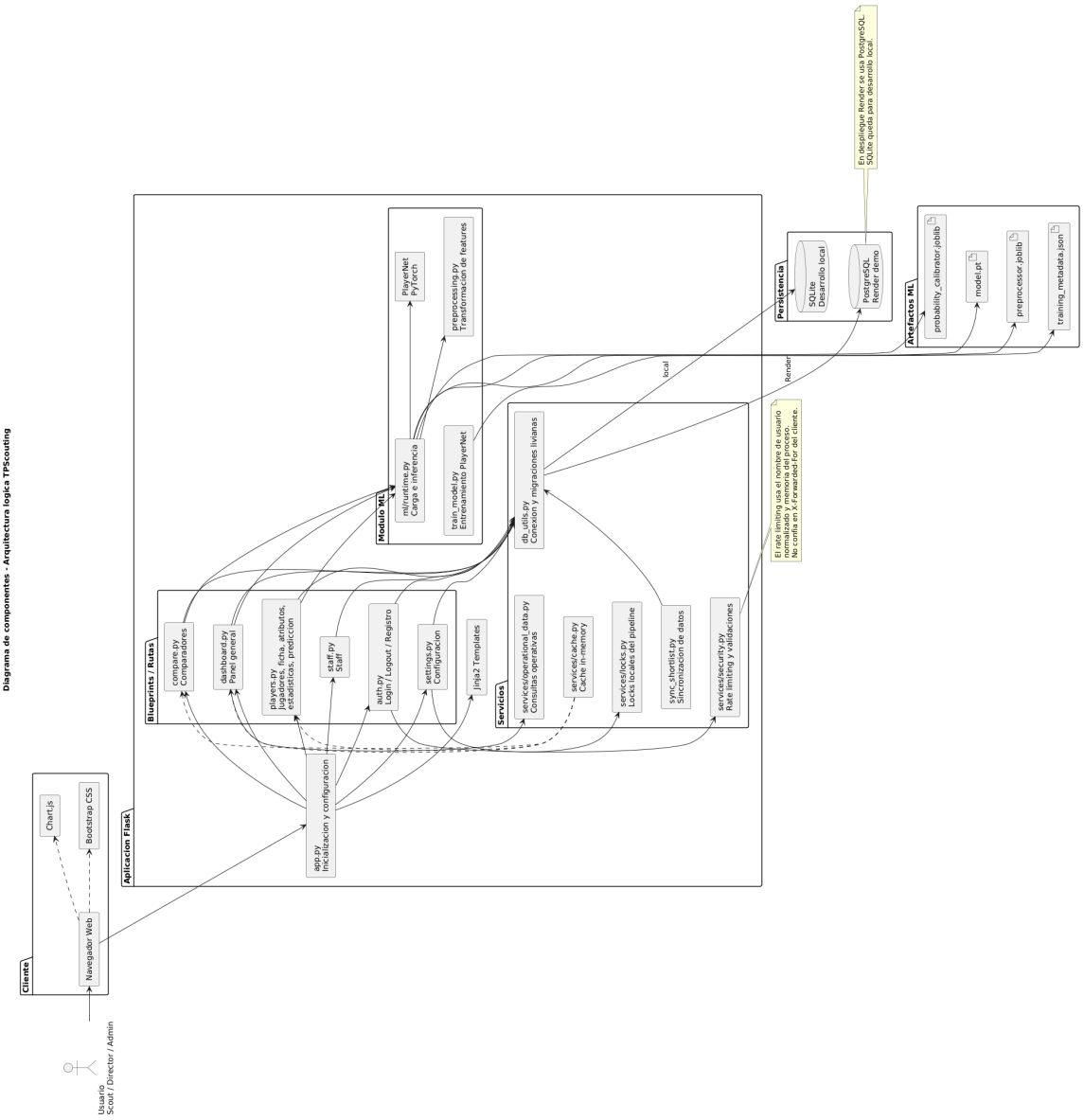


FIGURA 10-2. DIAGRAMA DE COMPONENTES DE TPSCOUTING AMPLIADO.

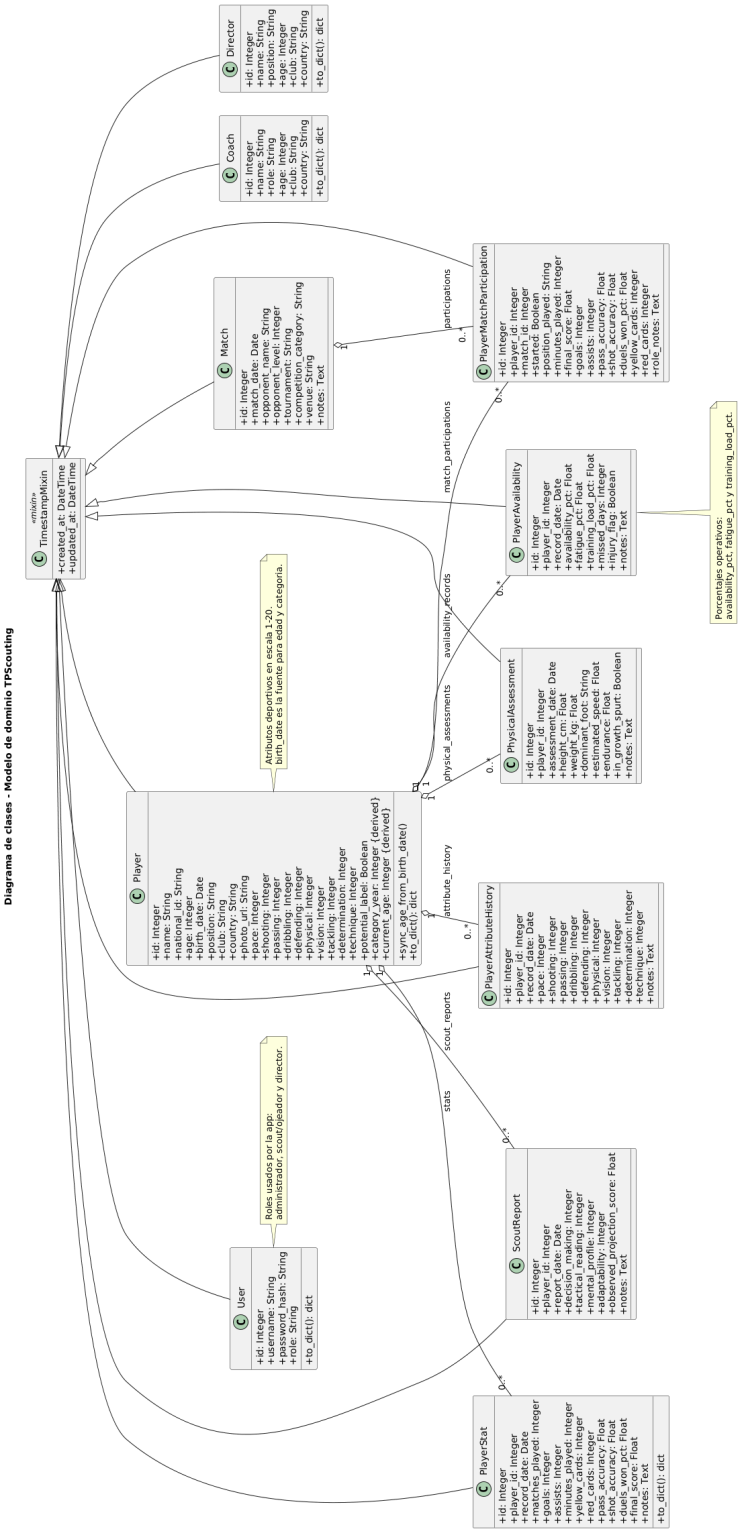


FIGURA 10-3. DIAGRAMA DE CLASES DEL MODELO DE DOMINIO AMPLIADO.

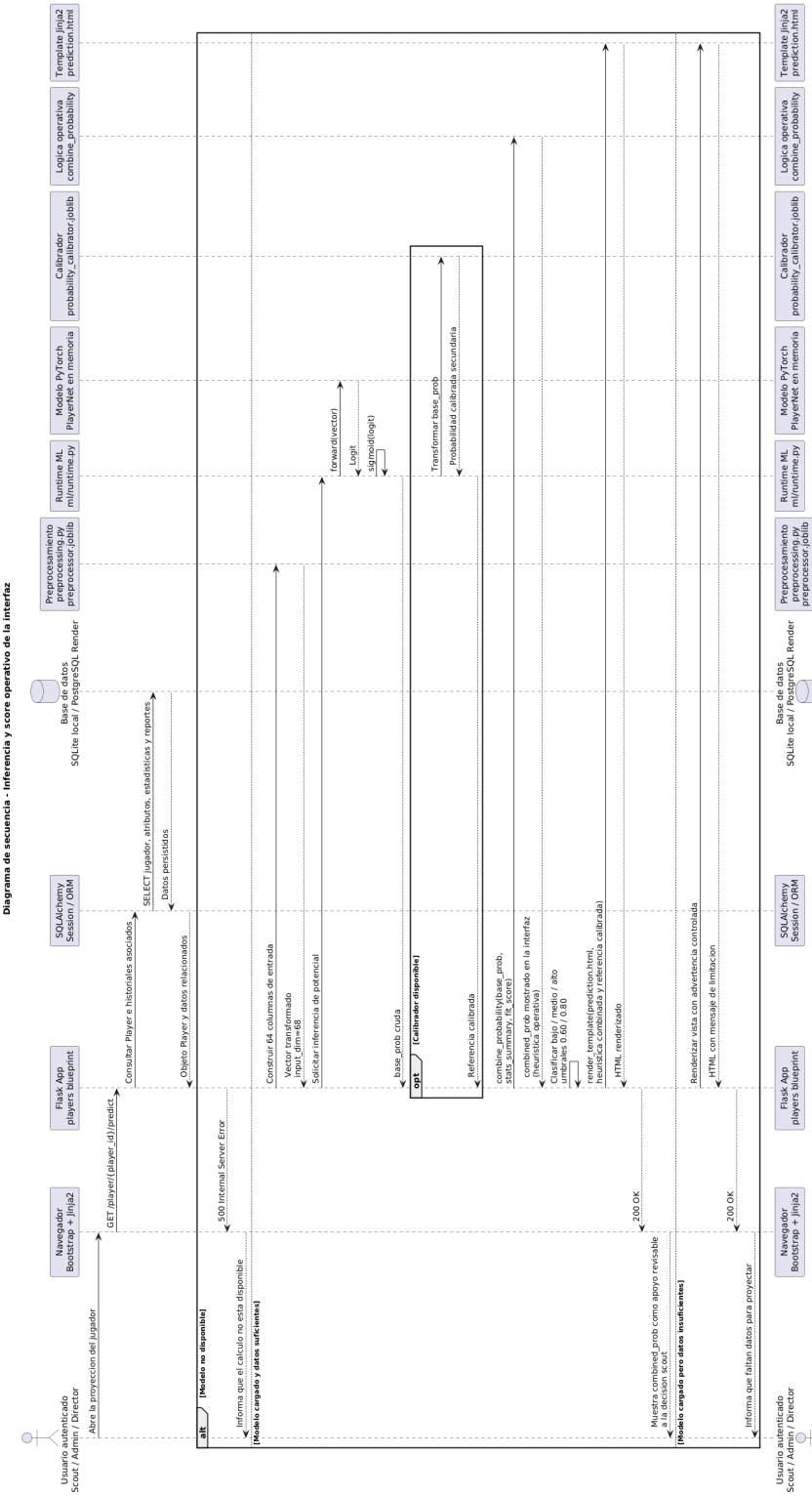


FIGURA 10-4. DIAGRAMA DE SECUENCIA DE INFERENCIA AMPLIADO.

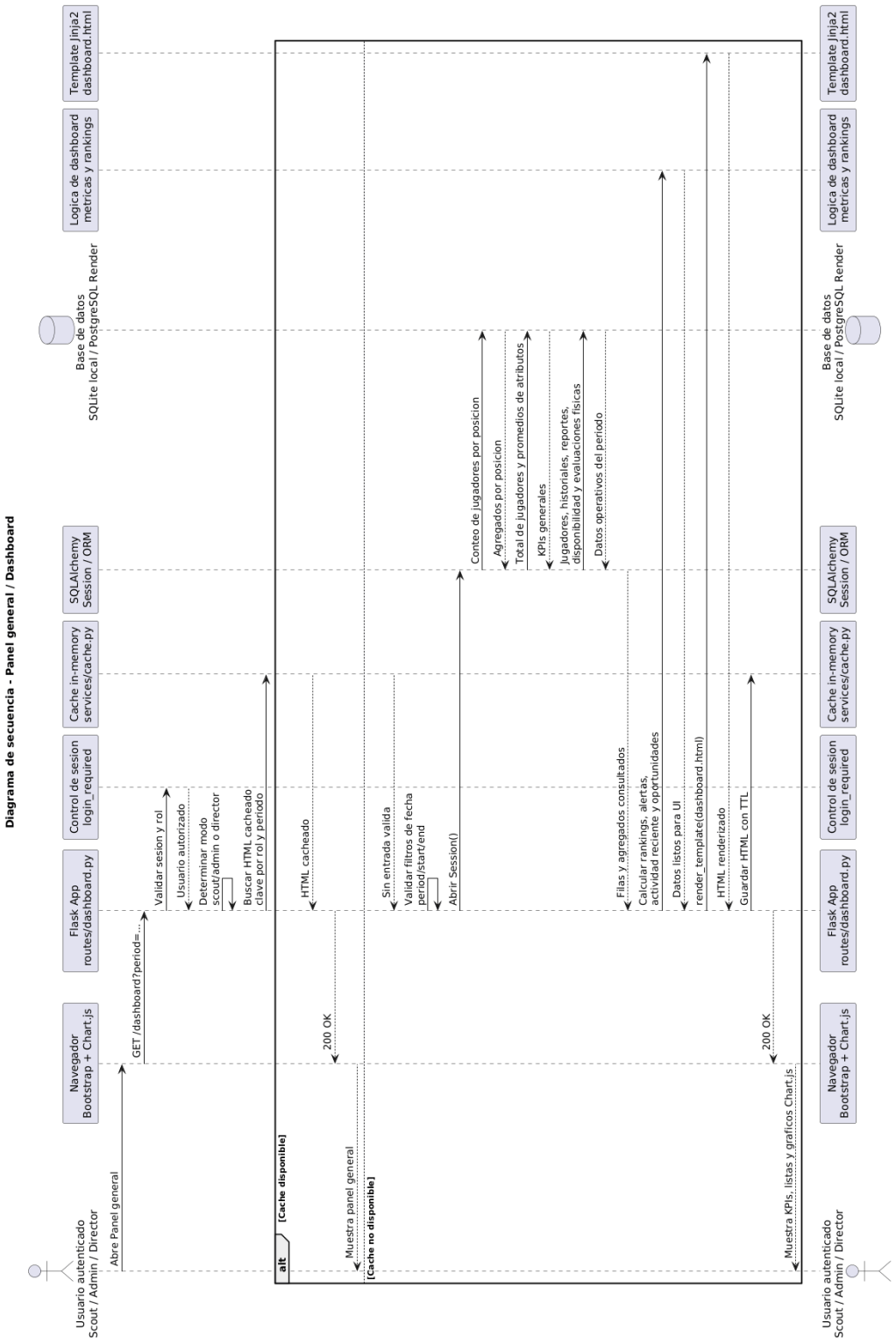


FIGURA 10-5. DIAGRAMA DE SECUENCIA DEL PANEL GENERAL AMPLIADO.

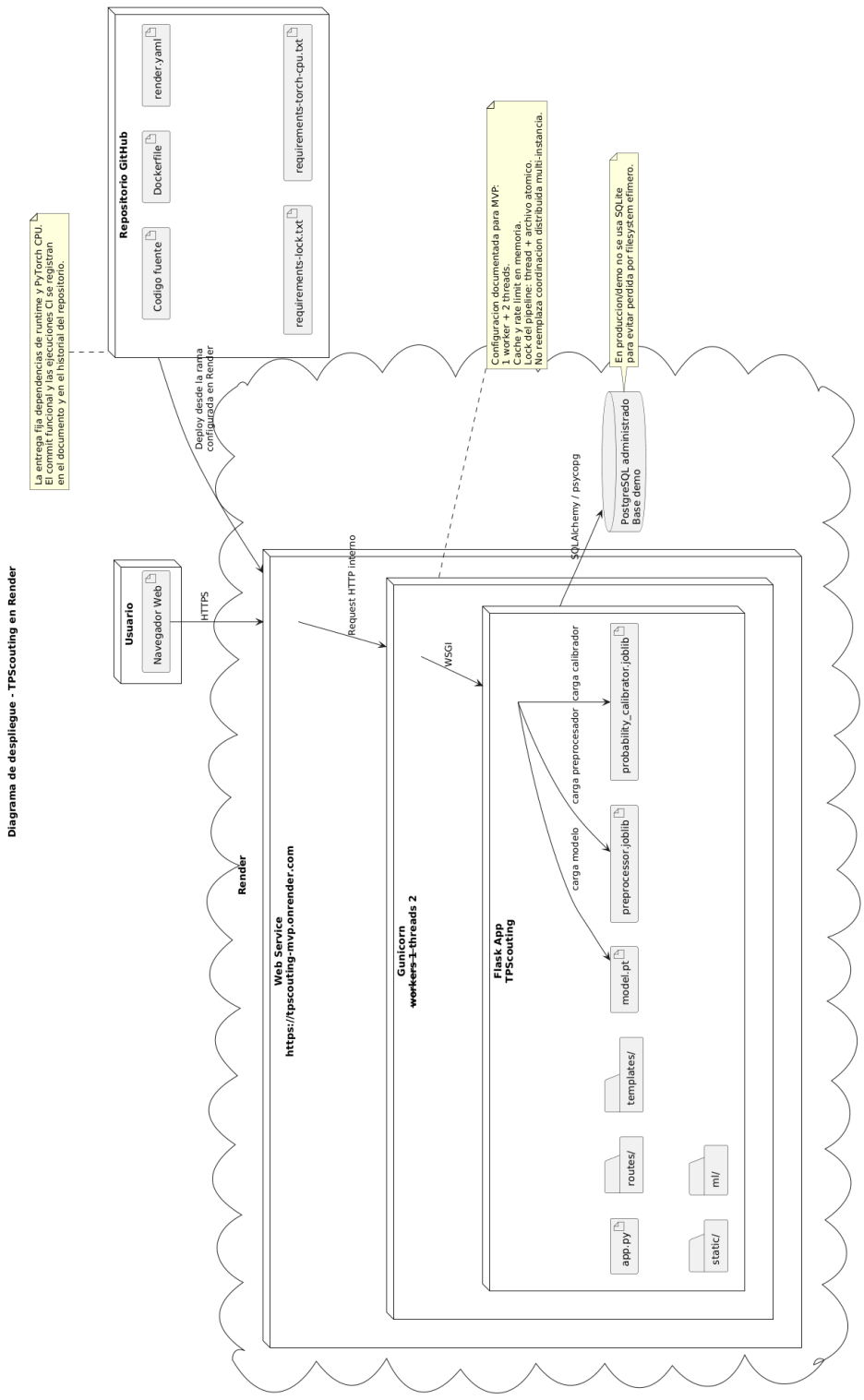


FIGURA 10-6. DIAGRAMA DE DESPLIEGUE EN RENDER AMPLIADO.

10.5 Casos de uso ampliados

Las Figuras 10-7 a 10-9 amplían los casos de uso generales, la gestión de jugadores y el análisis para decisión scout.

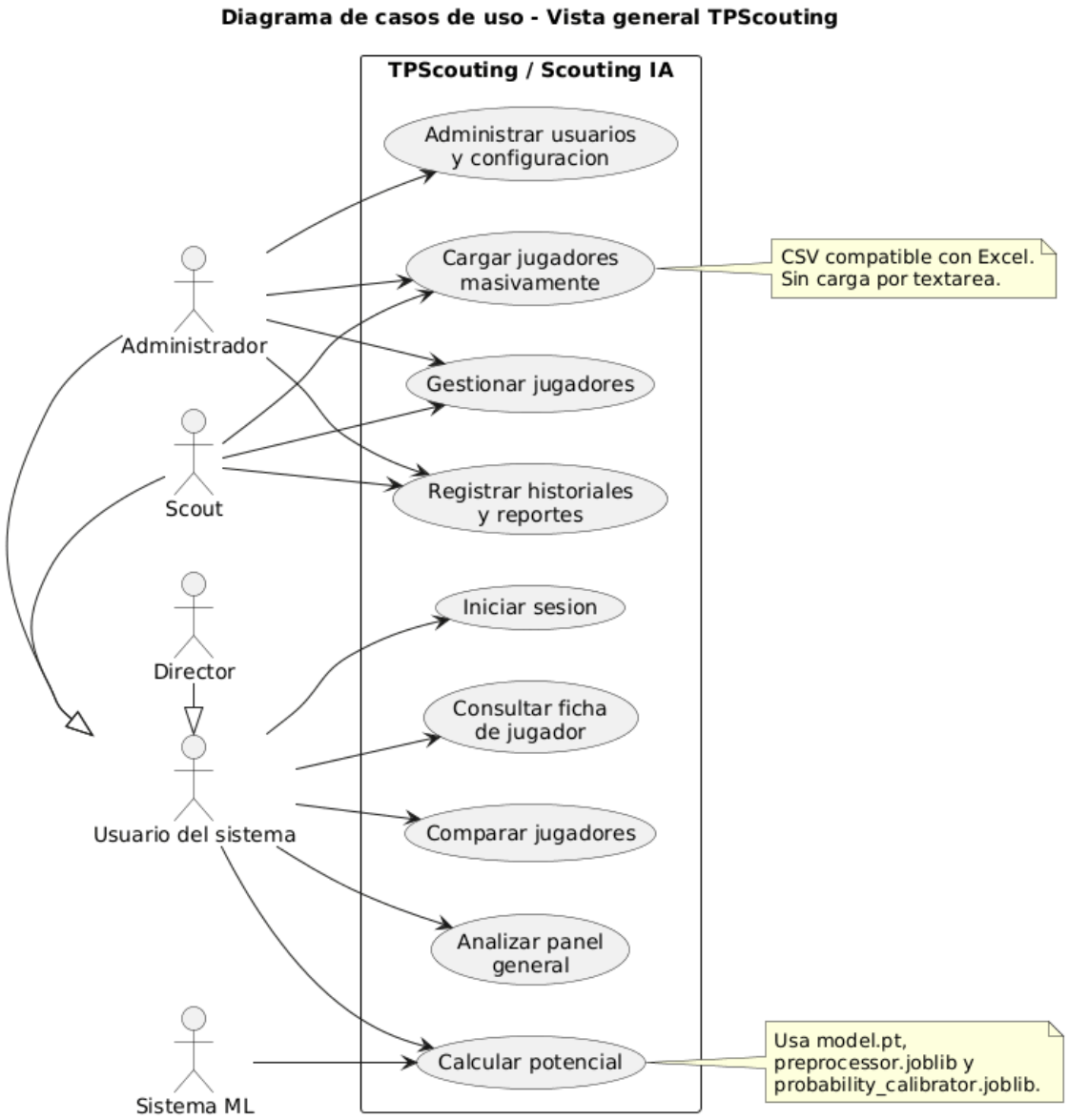


FIGURA 10-7. DIAGRAMA DE CASOS DE USO GENERAL AMPLIADO.

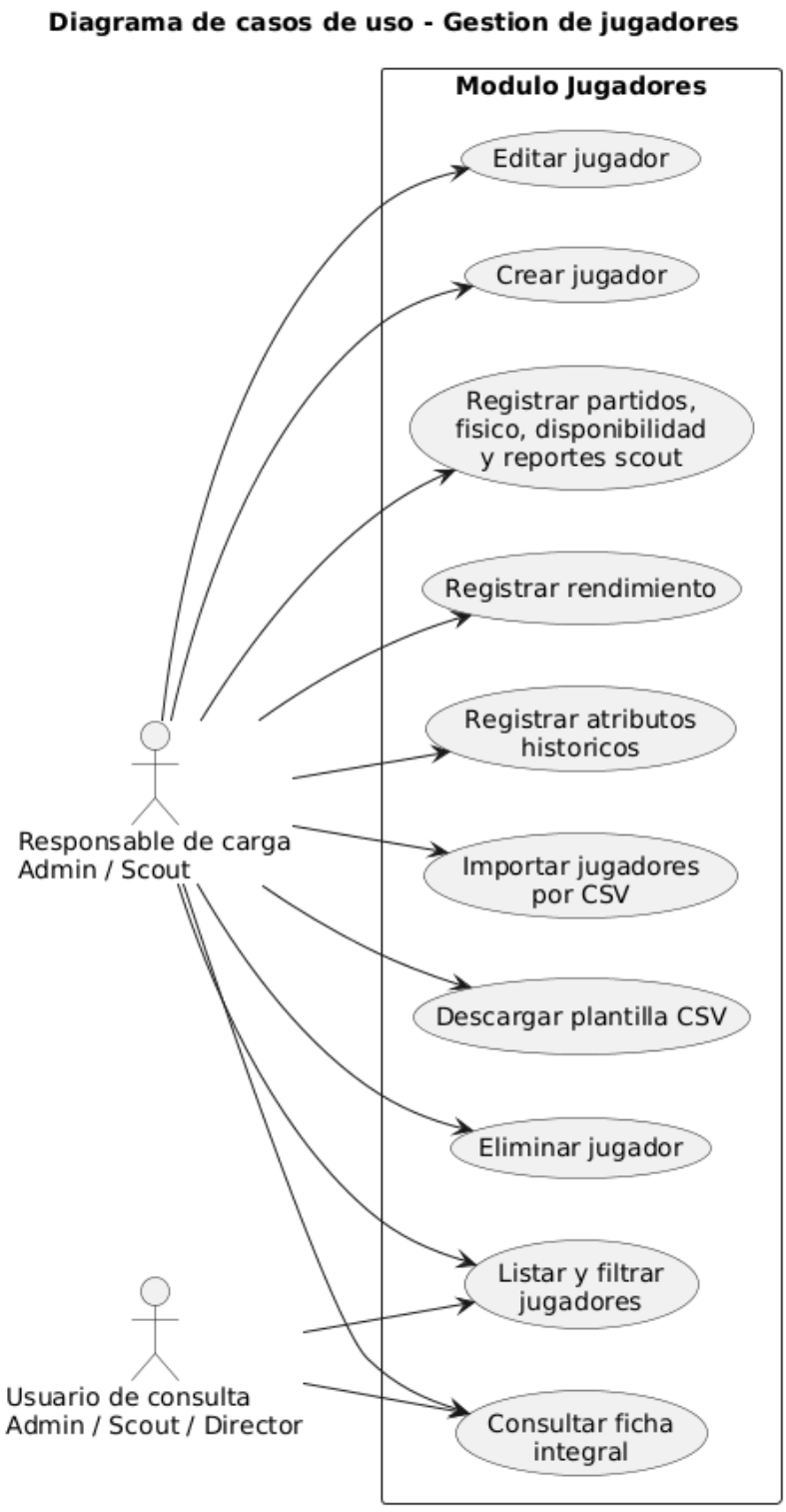


FIGURA 10-8. DIAGRAMA DE CASOS DE USO DE GESTIÓN DE JUGADORES AMPLIADO.

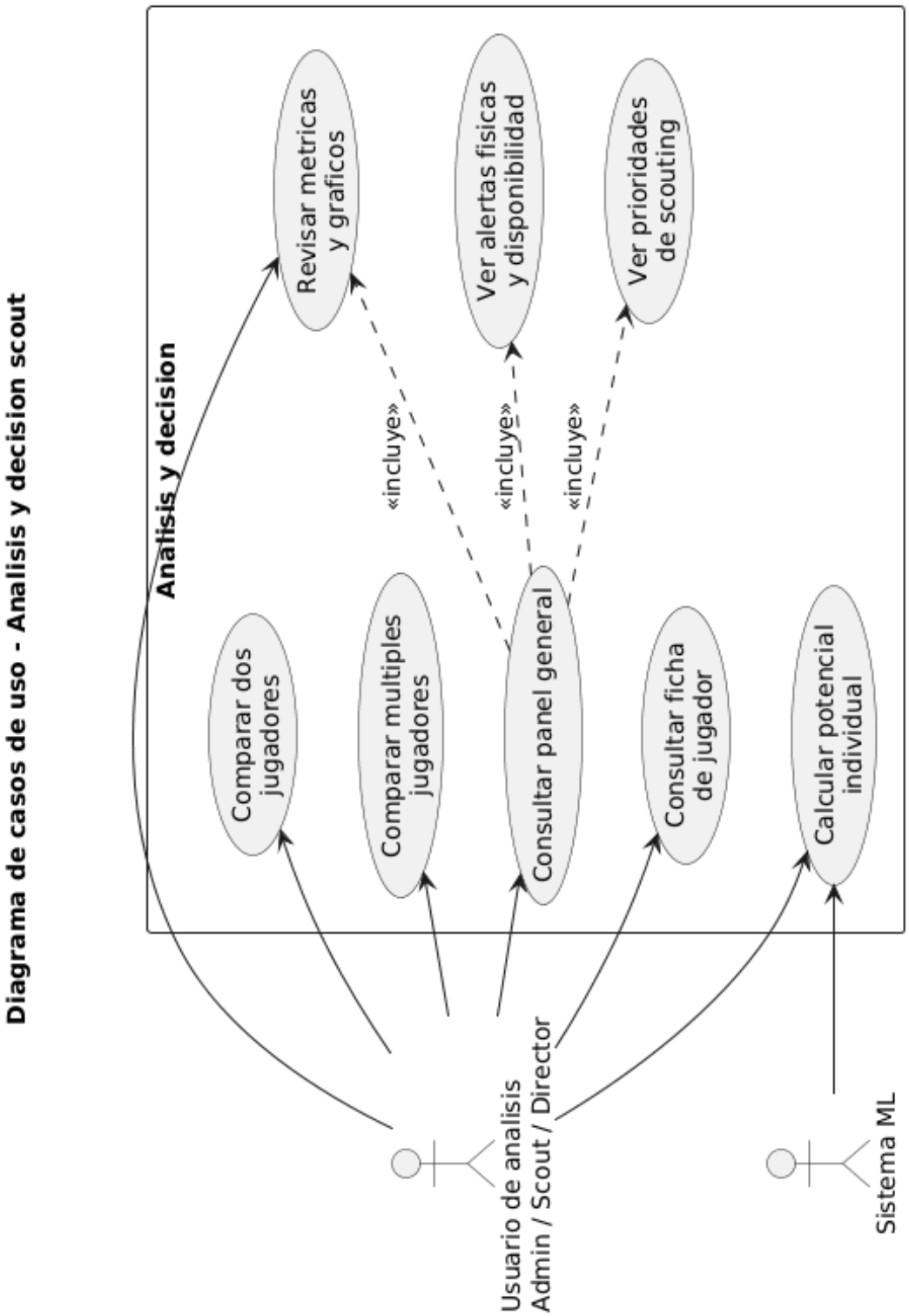


FIGURA 10-9. DIAGRAMA DE CASOS DE USO DE ANÁLISIS Y DECISIÓN SCOUT AMPLIADO.